

Méthodes de chargement de modules noyau Linux sans recompilation

15 mai 2026

Table des matières

1	Introduction	3
2	Vérifications effectuées lors du chargement d'un module	3
2.1	Vérification de la <code>CAP_SYS_MODULE</code>	3
2.2	Vérification de la signature du module	4
2.2.1	Configuration de la signature des modules	4
2.2.2	Gestion des clés de signature et signature manuelle des modules du noyau	5
2.2.3	Vue globale du fonctionnement de l'outil <code>sign-file</code>	6
2.2.4	Schéma du mécanisme de vérification des signatures des modules Linux	8
2.2.5	Mécanisme de vérification de la signature du module	9
2.3	Vérification de la compatibilité architecturale	17
2.4	Vérification de la version du noyau (<code>vermagic</code>)	17
2.5	Vérification de la structure <code>struct module</code>	19
2.6	Vérification des versions de symboles (CRC)	23
2.6.1	Mécanisme de vérification des versions de symboles	23
2.6.2	Infrastructure de versionnement par CRC des symboles	25
2.6.3	Génération des CRC avec <code>genksyms</code>	26
2.7	Vérification et résolution des symboles du module	29
2.8	Vérification et application des relocalisations ELF	33
3	Schémas du chargement et de la validation des modules noyau	40
3.1	Chaîne de chargement d'un module noyau après <code>insmod</code>	40
3.2	Structure interne d'un module compilé avec <code>CONFIG_MODVERSIONS</code>	41
3.3	Arbre des appels des fonctions des vérifications du noyau pour le chargement d'un module	42
4	Analyse d'un outil de modification de <code>vermagic</code> et CRC de module	43
4.1	Fonctionnement général	43
4.2	Preuve de concept : cas sans l'option <code>CONFIG_MODVERSIONS</code>	44
4.2.1	<code>Vermagic module = Vermagic noyau</code>	44
4.2.2	<code>Vermagic module > Vermagic noyau</code>	45
4.2.3	<code>Vermagic module < Vermagic noyau</code>	46
4.3	Preuve de concept : cas avec l'option <code>CONFIG_MODVERSIONS</code>	49
4.3.1	<code>Vermagic module = Vermagic noyau</code>	50
4.3.2	<code>Vermagic module != Vermagic noyau</code>	50

1 Introduction

Dans Linux, un module noyau (.ko) est un composant chargé dynamiquement pour ajouter une fonctionnalité (pilote, système de fichiers, etc.) sans recompiler le noyau. Les principales difficultés concernent la compatibilité entre le module et le noyau en cours d'exécution. En effet, un module compilé pour une version donnée du noyau ne peut généralement être chargé que sur cette même version.

Le module dépend de l'interface interne du noyau (symboles exportés, structures de données, options de compilation et architecture matérielle). Ainsi, même deux noyaux portant des numéros de version proches peuvent être incompatibles si leur configuration diffère.

Lors du chargement, le noyau vérifie ces informations et si elles ne correspondent pas, le module est refusé afin d'éviter des erreurs d'exécution, des corruptions mémoire ou un plantage du système.

Ce projet a pour objectif d'étudier les problématiques de compatibilité et de chargement des modules noyau sous Linux. Il vise notamment à comprendre dans quelles conditions un module binaire peut être chargé dynamiquement, sans recompilation, sur un noyau donné, ainsi que les limites imposées par les mécanismes de vérification du noyau.

L'étude s'articule autour des axes suivants :

- Vérifications effectuées lors du chargement d'un module
- Schémas du chargement et de la validation des modules noyau
- Analyse d'un outil de modification de `vermagic` et CRC de module

2 Vérifications effectuées lors du chargement d'un module

Forcer le chargement d'un module noyau binaire sans recompilation consiste à contourner les mécanismes de vérification mis en place par le noyau Linux. Les noyaux renforcent ces vérifications afin de limiter l'installation de modules malveillants, notamment les rootkits noyau. En pratique, un module noyau est étroitement dépendant de plusieurs éléments critiques du système :

- la version exacte du noyau ;
- sa configuration interne (options `CONFIG_*`) ;
- les symboles exportés par le noyau ;
- et parfois la clé de signature utilisée par *Secure Boot*.

Pour ces raisons, une recompilation est normalement nécessaire lorsqu'on change de noyau ou d'environnement. La suite de cette section présente les principales vérifications observées lors de l'analyse du code source du noyau.

2.1 Vérification de la `CAP_SYS_MODULE`

La capacité `CAP_SYS_MODULE` est un privilège critique du noyau Linux. Elle permet à un processus de charger et de décharger des modules noyau arbitraires, modifiant potentiellement le comportement du système. Une compromission de cette capacité peut mener à une élévation de privilèges, la prise de contrôle complète du système et le contournement des mécanismes de sécurité du noyau, notamment les modules de sécurité Linux (LSM) et les environnements de conteneurs. Après une telle compromission, il devient possible de charger des modules sans respecter les restrictions habituelles [1].

Le noyau vérifie cette capacité via la fonction `may_init_module()` :

```
static int may_init_module(void)
{
    if (!capable(CAP_SYS_MODULE) || modules_disabled)
        return -EPERM;

    return 0;
}
```

Cette fonction refuse le chargement si le processus ne possède pas la capacité `CAP_SYS_MODULE` ou si le chargement de modules est désactivé (`modules_disabled`).

2.2 Vérification de la signature du module

Le mécanisme de signature des modules du noyau signe cryptographiquement les modules lors de leur installation, puis vérifie la signature lors du chargement du module. Cela permet d'augmenter la sécurité du noyau en empêchant le chargement de modules non signés ou de modules signés avec une clé invalide. La signature des modules renforce la sécurité en rendant plus difficile le chargement d'un module malveillant dans le noyau.

Ce mécanisme utilise des certificats standard X.509 pour encoder les clés publiques utilisées lors de la vérification des signatures. Historiquement, le mécanisme utilisait uniquement l'algorithme de chiffrement asymétrique RSA avec des clés de 4096 bits, un choix très courant pour la signature des modules du noyau. Les noyaux récents prennent également en charge ECDSA. Les algorithmes de hachage possibles sont SHA-1, SHA-224, SHA-256, SHA-384 et SHA-512.

2.2.1 Configuration de la signature des modules

La signature des modules du noyau est configurée via les options de compilation du noyau Linux. Elle permet de contrôler la manière dont les modules sont signés et vérifiés lors de leur chargement.

Cette configuration inclut notamment l'activation du mécanisme de vérification des signatures, la gestion du comportement du noyau face aux modules non signés ou invalide, ainsi que définir l'algorithme cryptographique utilisé pour produire les signatures.

L'ensemble de ces options est accessible dans la section `Enable loadable module support` de la configuration du noyau.

```
[*] Enable loadable module support --->
[*]   Module signature verification
[*]     Require modules to be validly signed
[*]     Automatically sign all modules
      Which hash algorithm should modules be signed with? (Sign modules with SHA-512) --->
```

L'option de configuration `CONFIG_MODULE_SIG` active la vérification des signatures des modules du noyau lors de leur chargement. Lorsqu'elle est activée, le noyau vérifie qu'une signature valide est présente sur chaque module chargé. Cette signature est simplement ajoutée à la fin du fichier du module.

Cette option introduit une dépendance aux bibliothèques de développement OpenSSL lors de la compilation du noyau, afin que l'outil de signature puisse utiliser ses primitives cryptographiques.

Il est recommandé d'activer cette option si l'on utilise `SECURITY_LOCKDOWN_LSM` ou un autre mécanisme de verrouillage (lockdown) via un LSM, car sans cela les modules non signés peuvent être chargés malgré les politiques de sécurité [2].

L'option `Require modules to be validly signed` correspond à `CONFIG_MODULE_SIG_FORCE`. Cette option définit le comportement du noyau face aux modules dont la signature est invalide ou absente. Lorsque cette option est désactivée (mode dit *permissif*), les modules non signés ou dont la clé de signature est inconnue peuvent être chargés. Toutefois, le noyau est alors marqué comme *tainted* (altéré), et les modules concernés sont également signalés comme tels, notamment par le caractère E.

Lorsque cette option est activée (mode dit *restrictif*), seuls les modules disposant d'une signature valide, vérifiable à l'aide d'une clé publique présente dans le noyau, peuvent être chargés. Tout autre module est rejeté avec une erreur. Indépendamment de ce paramètre, tout module contenant une section de signature invalide ou non interprétable est systématiquement rejeté.

L'option `Automatically sign all modules` correspond à `CONFIG_MODULE_SIG_ALL`. Lorsque cette option est activée, les modules sont automatiquement signés lors de l'étape `modules_install` du processus de compilation du noyau. Cela permet d'assurer que tous les modules installés disposent d'une signature valide sans intervention manuelle.

Lorsque cette option est désactivée, la signature des modules doit être effectuée manuellement à l'aide de l'outil `scripts/sign-file`, fourni avec les sources du noyau.

Il est nécessaire de sélectionner l'algorithme de hachage utilisé pour la signature cryptographique des modules. Dans l'exemple présenté, l'algorithme `SHA-512` est utilisé.

L'algorithme choisi est directement intégré au noyau. Cela permet de vérifier les signatures des modules utilisant cet algorithme sans créer de dépendance lors du chargement des modules.

2.2.2 Gestion des clés de signature et signature manuelle des modules du noyau

Des paires de clés cryptographiques sont nécessaires pour générer et vérifier les signatures. Une clé privée est utilisée pour générer une signature et la clé publique correspondante est utilisée pour la vérifier. La clé privée n'est nécessaire que pendant la compilation, après quoi elle peut être supprimée ou stockée de manière sécurisée. La clé publique est intégrée dans le noyau afin de pouvoir vérifier les signatures lors du chargement des modules.

Dans des conditions normales, lorsque l'option `CONFIG_MODULE_SIG_KEY` n'est pas modifiée par rapport à sa valeur par défaut, le processus de compilation du noyau génère automatiquement une paire de clés à l'aide d'OpenSSL si aucune clé n'est présente dans le fichier :

```
certs/signing_key.pem
```

Cette génération intervient lors de la construction de `vmlinux`, la partie publique de la clé étant intégrée directement dans le noyau. Les paramètres de génération sont définis dans le fichier :

```
certs/x509.genkey
```

Pour plus d'informations, il est possible de se référer à la documentation officielle [3].

Par ailleurs, il est possible de générer manuellement une paire de clés. Le chemin complet du fichier `kernel-key.pem` ainsi obtenu peut ensuite être spécifié dans l'option `CONFIG_MODULE_SIG_KEY`. Le certificat et la clé qu'il contient seront alors utilisés à la place de la paire de clés générée automatiquement.

Afin de permettre la vérification des signatures, le noyau intègre un trousseau de clés publiques (en anglais *keyring*) accessible par l'utilisateur `root`. Ces clés sont stockées dans un keyring nommé `.builtin_trusted_keys`, consultable via la commande suivante :

```
root@vm:~$ cat /proc/keys
...
223c7853 I-----      1 perm 1f030000      0      0 keyring   .builtin_trusted_keys: 1
302d2d52 I-----      1 perm 1f010000      0      0 asymmetri Fedora kernel signing key:
↳ d69a84e6bce3d216b979e9505b3e3ef9a7118079: X509.RSA a7118079 []
...
```

Dans ce contexte, les modules du noyau incluent une signature numérique ajoutée à la fin du fichier. Il est possible de vérifier la présence de cette signature à l'aide d'un simple `hexdump` [4] :

```
user@:~$hexdump -C my_module.ko | tail
00008880  cf 0e e7 cb 10 9e 98 5f 4b 21 d4 03 ba 3d 7e e7 |....._K!...=~.|
00008890  68 db f9 e3 5f 62 3c c7 d6 6c 84 c7 d6 68 c1 73 |h..._b<..l...h.s|
000088a0  3d d7 5a 38 66 99 12 b8 84 c9 84 45 dd 68 6d 17 |=.Z8f.....E.hm.|
000088b0  03 24 dc 9c 6f 6d 11 01 e9 74 82 ea b5 5b 46 07 |.$.om...t...[F.|
000088c0  fe dd 66 97 1a 33 58 3d 6e d0 ac 03 08 16 73 06 |..f..3X=n.....s.|
000088d0  9f 90 c4 eb b3 82 1d 9f 48 8c 5b 51 01 06 01 1e |.....H.[Q....|
000088e0  14 00 00 00 00 00 02 02 7e 4d 6f 64 75 6c 65 20 |.....~Module |
000088f0  73 69 67 6e 61 74 75 72 65 20 61 70 70 65 6e 64 |signature append|
00008900  65 64 7e 0a                                     |ed~.|
00008904
```

On observe ici la présence de la chaîne `Module signature appended` en fin de fichier, ce qui indique qu'une signature est bien attachée au module.

Toutefois, cette observation permet uniquement de confirmer la présence d'une signature, sans en garantir la validité. La vérification effective de celle-ci est réalisée par le noyau lors du chargement du module, en s'appuyant sur les clés contenues dans le keyring de confiance.

Il est également possible d'afficher les informations relatives à la signature avec :

```
root@vm:~$ modinfo my_module.ko | grep '^sig'
signer:      Modules
sig_key:     B0:3B:5E:DB:57:00:F9:D5:D7:85:EB:2D:6F:3E:19:D3:4A:20:20:5B
sig_hashalgo: sha512
```

La clé privée, stockée dans le fichier `certs/signing_key.pem`, doit être déplacée vers un emplacement sécurisé (sauf si de nouvelles clés doivent être générées pour chaque noyau, auquel cas ce fichier peut être supprimé).

Il est fortement déconseillé de conserver cette clé dans `/usr/src/linux` sur un système en production, car un logiciel malveillant pourrait l'utiliser pour signer des modules noyau malveillants (tels que des rootkits), compromettant ainsi davantage le système.

Une bonne pratique consiste à stocker la clé privée sur un support externe, comme une clé USB, qui sera déconnectée après utilisation. Ainsi, lors de la mise à jour de certains pilotes (par exemple les pilotes graphiques) ne nécessitant pas une recompilation complète du noyau, il est possible de connecter temporairement ce support afin de signer les modules, puis de le retirer immédiatement après.

D'autres solutions existent, comme l'utilisation de technologies matérielles telles que le TPM (*Trusted Platform Module*). Toutefois, celles-ci sont parfois controversées, car le module TPM est généralement fourni avec une clé intégrée dès sa fabrication, sur laquelle le fabricant conserve un certain contrôle.

Dans certains cas, il peut être nécessaire de signer manuellement un module du noyau. Pour cela, il est possible d'utiliser le script `scripts/sign-file`, disponible dans l'arborescence des sources du noyau Linux [4]. Ce script requiert quatre arguments :

- L'algorithme de hachage à utiliser, par exemple `sha512`.
- L'emplacement de la clé privée.
- L'emplacement du certificat (contenant la clé publique).
- Le module du noyau à signer.

Exemple d'utilisation :

```
root@vm:~$ /usr/src/linux/scripts/sign-file sha512 /usr/src/linux/certs/signing_key.pem
↪ /usr/src/linux/certs/signing_key.x509 my_module.ko
```

Par défaut, les certificats sont situés dans le répertoire `certs` de l'arborescence des sources du noyau.

2.2.3 Vue globale du fonctionnement de l'outil `sign-file`

L'outil `sign-file` est utilisé dans le noyau Linux afin de signer cryptographiquement un module kernel. Il ajoute une signature numérique à la fin du fichier du module afin que le noyau puisse vérifier que celui-ci provient d'une source autorisée et qu'il n'a pas été modifié après sa signature.

L'outil `sign-file` utilise CMS avec OpenSSL 1.0.0 ou plus récent. Sinon, il utilise PKCS#7. Ce format plus ancien limite le contrôle sur le certificat X.509 et ne permet pas d'ajouter manuellement des signataires dans un message PKCS#7. Il faut donc accepter qu'un certificat soit automatiquement inclus dans le message de signature. Enfin, ces anciennes versions d'OpenSSL ne supportent qu'un ensemble limité d'algorithmes de signature, principalement SHA1, qui devient alors obligatoire.

Le programme commence par la lecture de la clé privée utilisée pour signer le module. Deux types de sources de clés privées sont supportés : une clé stockée dans un fichier PEM ou une clé accessible via PKCS#11.

PKCS#11 est une norme de cryptographie à clé publique permettant de créer et de manipuler des jetons cryptographiques contenant des clés secrètes. Cette norme est fréquemment utilisée pour communiquer avec des modules matériels de sécurité (HSM) ou des cartes à puce [5].

Le programme lit ensuite le certificat X.509 associé à la clé privée. Afin de déterminer son format, il analyse les deux premiers octets du fichier. Deux encodages principaux existent pour les certificats X.509 :

- PEM : format texte encodé en Base64 ;
- DER : format binaire.

L'outil utilise ensuite la bibliothèque **OpenSSL**, une bibliothèque open source largement utilisée pour manipuler les certificats X.509 et les clés cryptographiques [6].

Après le chargement de la clé privée et du certificat, le programme calcule l'empreinte cryptographique (*hash*) du module à l'aide de l'algorithme demandé, par exemple SHA-256 ou SHA-512. Il génère ensuite un message au format PKCS#7/CMS contenant la signature cryptographique, les informations sur l'algorithme de hachage et le certificat X.509 du signataire.

Le programme crée alors un nouveau fichier contenant successivement le module ELF original, le message PKCS#7, une structure `module_signature` ainsi que la chaîne magique " Module signature appended " indiquant la présence d'une signature.

La structure `module_signature` est définie comme suit :

```
struct module_signature {
    uint8_t      algo;           /* Public-key crypto algorithm [0] */
    uint8_t      hash;          /* Digest algorithm [0] */
    uint8_t      id_type;       /* Key identifier type [PKEY_ID_PKCS7] */
    uint8_t      signer_len;    /* Length of signer's name [0] */
    uint8_t      key_id_len;    /* Length of key identifier [0] */
    uint8_t      __pad[3];
    uint32_t      sig_len;      /* Length of signature data */
};
```

Tous les champs de cette structure ne sont pas nécessairement utilisés. Dans l'implémentation actuelle de `sign-file`, seuls les champs `id_type` et `sig_len` sont réellement renseignés. Les autres champs restent à zéro car la majorité des informations cryptographiques sont déjà présentes dans le blob ASN.1 PKCS#7.

La structure finale d'un module signé est donc la suivante :

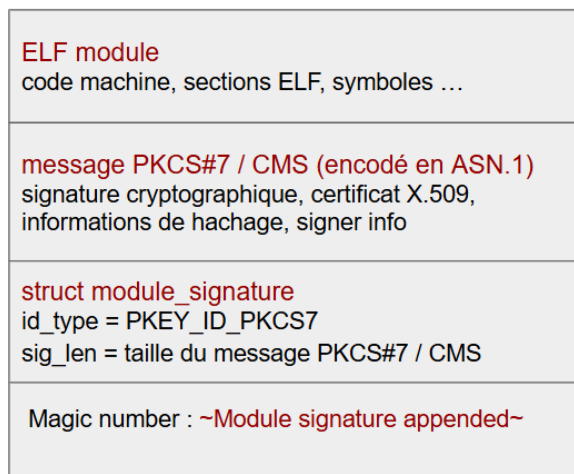


FIGURE 1 – Structure interne simplifiée d'un module signé compilé

2.2.4 Schéma du mécanisme de vérification des signatures des modules Linux

Le schéma suivant présente une vue d'ensemble du mécanisme de vérification des signatures des modules du noyau Linux. Il illustre les différentes étapes effectuées lors du chargement d'un module signé, depuis la détection de la signature jusqu'à la validation cryptographique et l'établissement de la chaîne de confiance X.509 via les *keyrings* du noyau Linux.

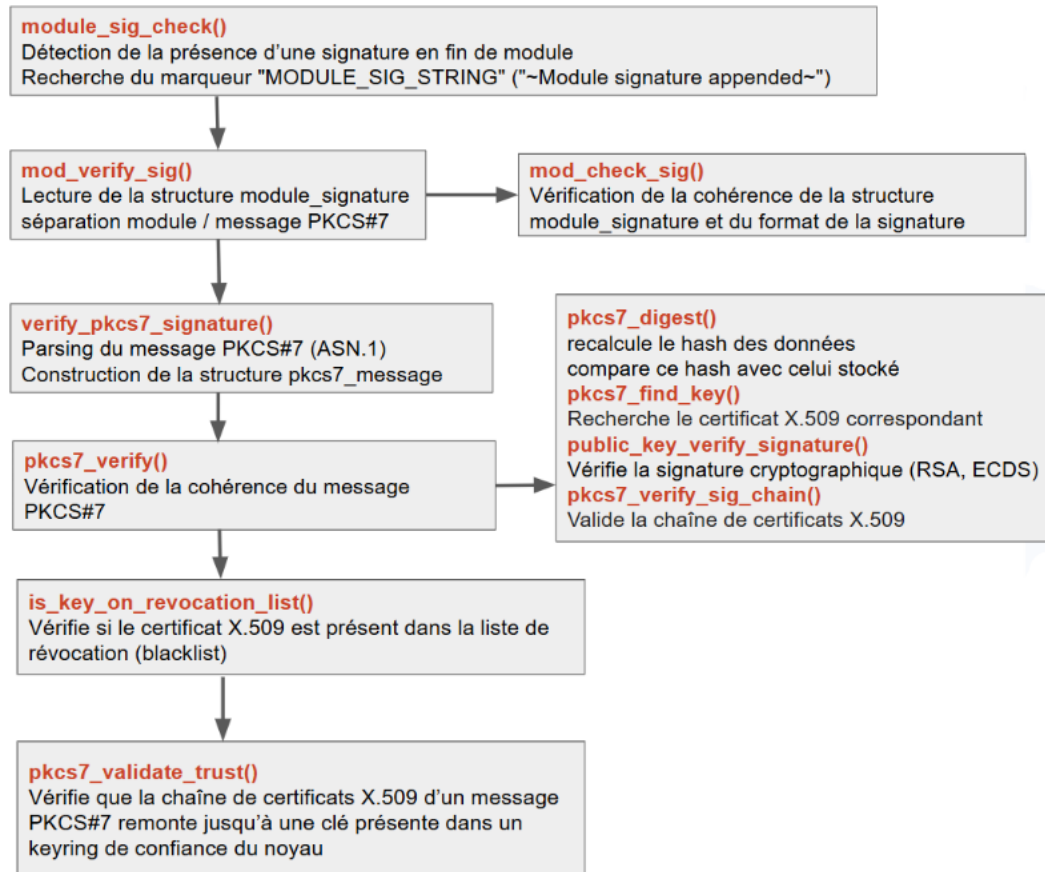


FIGURE 2 – Mécanisme de vérification des signatures PKCS#7 des modules Linux

Le schéma précédent résume les principales vérifications effectuées par le noyau Linux lors du chargement d'un module signé :

- **Détection de la signature** : Le noyau recherche le marqueur `MODULE_SIG_STRING` en fin de fichier afin de déterminer si une signature est attachée au module.
- **Extraction des informations de signature** : La structure `module_signature` est récupérée afin d'accéder aux métadonnées nécessaires à l'extraction et à l'interprétation de la signature PKCS#7.
- **Validation du format PKCS#7** : Le noyau vérifie la cohérence des informations de signature avant toute opération cryptographique.
- **Décodage ASN.1 du message PKCS#7** : Le message PKCS#7 est parsé afin de construire les structures internes contenant les certificats X.509, les signatures et les informations cryptographiques nécessaires à la vérification.
- **Vérification cryptographique** : Le noyau recalcule le hash des données signées puis vérifie la signature numérique à l'aide de la clé publique du certificat signataire.
- **Validation de la chaîne de certificats X.509** : La chaîne de confiance est vérifiée afin de garantir que chaque certificat est correctement signé jusqu'à atteindre une autorité reconnue par le noyau.
- **Validation via les *keyrings* du noyau** : Le certificat du signataire ou le certificat racine est recherché dans les *keyrings* de confiance du noyau (`.builtin_trusted_keys`, `.secondary`, `.platform`, etc.).

- **Acceptation ou rejet du module** : Le module est chargé uniquement si l'ensemble des vérifications cryptographiques et des contrôles de confiance réussissent.

2.2.5 Mécanisme de vérification de la signature du module

Lorsque le noyau est compilé avec `CONFIG_MODULE_SIG`, la signature des modules est vérifiée au moment du chargement pour garantir authenticité et intégrité.

Tout d'abord, la fonction `module_sig_check()`, située dans `linux/kernel/module/signing.c`, est exécutée :

```
int module_sig_check(struct load_info *info, int flags) {
    int err = -ENODATA;
    const unsigned long markerlen = sizeof(MODULE_SIG_STRING) - 1;
    const char *reason;
    const void *mod = info->hdr;
    bool mangled_module =
        flags & (MODULE_INIT_IGNORE_MODVERSIONS | MODULE_INIT_IGNORE_VERMAGIC);
    /*
     * Do not allow mangled modules as a module with version information
     * removed is no longer the module that was signed.
     */
    if (!mangled_module && info->len > markerlen &&
        memcmp(mod + info->len - markerlen, MODULE_SIG_STRING, markerlen) == 0) {
        /* We truncate the module to discard the signature */
        info->len -= markerlen;
        err = mod_verify_sig(mod, info);
        if (!err) {
            info->sig_ok = true;
            return 0;
        }
    }

    /*
     * We don't permit modules to be loaded into the trusted kernels
     * without a valid signature on them, but if we're not enforcing,
     * certain errors are non-fatal.
     */
    switch (err) {
    case -ENODATA:
        reason = "unsigned module";
        break;
    case -ENOPKG:
        reason = "module with unsupported crypto";
        break;
    case -ENOKEY:
        reason = "module with unavailable key";
        break;
    default:
        /*
         * All other errors are fatal, including lack of memory,
         * unparseable signatures, and signature check failures --
         * even if signatures aren't required.
         */
        return err;
    }

    if (is_module_sig_enforced()) {
        pr_notice("Loading of %s is rejected\n", reason);
        return -EKEYREJECTED;
    }

    return security_locked_down(LOCKDOWN_MODULE_SIGNATURE);
}
```

```
}

```

Comme dit plus haut, cette fonction sert à vérifier authenticité et intégrité du module via une signature cryptographique. Son objectif principal est d'empêcher le chargement de modules :

- non signés,
- modifiés après signature,
- signés avec une clé inconnue ou un algorithme non supporté,

Elle initialise un code d'erreur indiquant par défaut l'absence de signature, puis vérifie si le module a été altéré par l'ignorance de certaines informations de version lors du chargement, ce qui invaliderait sa signature.

Si le module est intact et de taille suffisante, la fonction contrôle la présence du marqueur `MODULE_SIG_STRING` en fin de fichier, indiquant qu'une signature y est attachée. Le cas échéant, cette signature est exclue du module et la fonction `mod_verify_sig()` est appelée pour en vérifier la validité cryptographique; en cas de succès, le module est considéré comme valide et son chargement est autorisé. En cas d'échec, des erreurs telles que l'absence de signature, un algorithme non supporté ou une clé indisponible sont distinguées, tandis que toute autre erreur entraîne un rejet immédiat.

Enfin, si la politique de sécurité du noyau impose la vérification des signatures, tout module non signé ou invalide est rejeté, sinon la décision dépend du mécanisme de verrouillage de sécurité actif.

```
int mod_verify_sig(const void *mod, struct load_info *info) {
    struct module_signature ms;
    size_t sig_len, modlen = info->len;
    int ret;

    pr_devel("==>%s(,%zu)\n", __func__, modlen);

    if (modlen <= sizeof(ms))
        return -EBADMSG;

    memcpy(&ms, mod + (modlen - sizeof(ms)), sizeof(ms));

    ret = mod_check_sig(&ms, modlen, "module");
    if (ret)
        return ret;

    sig_len = be32_to_cpu(ms.sig_len);
    modlen -= sig_len + sizeof(ms);
    info->len = modlen;

    return verify_pkcs7_signature(mod, modlen, mod + modlen, sig_len,
                                  VERIFY_USE_SECONDARY_KEYRING,
                                  VERIFYING_MODULE_SIGNATURE, NULL, NULL);
}
```

Elle commence par s'assurer que la taille du module est suffisante pour contenir une structure de signature, puis copie cette dernière depuis la fin du module. La fonction `mod_check_sig()` est ensuite appelée afin de valider le format et les informations de la signature; en cas d'erreur, celle-ci est immédiatement renvoyée.

Si cette vérification réussit, la longueur réelle de la signature est extraite, permettant d'ajuster la taille du module pour exclure les données de signature. Enfin, la fonction `verify_pkcs7_signature()` est appelée afin de vérifier l'authenticité de la signature PKCS#7 à l'aide des clés présentes dans le trousseau secondaire du noyau, déterminant ainsi si le module peut être considéré comme fiable.

Analysons la fonction `mod_check_sig()` :

```
int mod_check_sig(const struct module_signature *ms, size_t file_len,
                  const char *name) {
    if (be32_to_cpu(ms->sig_len) >= file_len - sizeof(*ms))
        return -EBADMSG;

    if (ms->id_type != PKEY_ID_PKCS7) {
```

```

pr_err("%s: not signed with expected PKCS#7 message\n", name);
return -ENOPKG;
}

if (ms->algo != 0 || ms->hash != 0 || ms->signer_len != 0 ||
    ms->key_id_len != 0 || ms->__pad[0] != 0 || ms->__pad[1] != 0 ||
    ms->__pad[2] != 0) {
pr_err("%s: PKCS#7 signature info has unexpected non-zero params\n", name);
return -EBADMSG;
}

return 0;
}

```

Cette vérification se fait en trois étapes principales :

- Vérification que la taille de la signature ne dépasse pas celle du fichier, pour éviter tout dépassement de tampon.
- Acceptation uniquement du format PKCS#7, réduisant la surface d'attaque.
- Contrôle que les paramètres inutilisés sont à 0 ; toute valeur non nulle est suspecte.

Avant d'analyser le comportement de la fonction `verify_pkcs7_signature`, il convient de présenter le standard PKCS#7.

Parmi les différents standards PKCS (*Public-Key Cryptography Standards*), le standard PKCS#7 occupe une place importante dans les systèmes modernes, notamment dans le noyau Linux. Également connu sous le nom de *Cryptographic Message Syntax* (CMS), PKCS#7 définit un format permettant d'encapsuler des données signées ainsi que les certificats X.509 associés. Les algorithmes RSA et ECDSA sont deux algorithmes de signature pouvant être utilisés dans le cadre de cette norme.

Dans le noyau Linux, ce mécanisme est utilisé afin d'assurer la vérification de l'intégrité et de l'authenticité des modules du noyau. Avant d'étudier le mécanisme de vérification des signatures PKCS#7, il est nécessaire de présenter la structure `struct pkcs7_message` :

```

struct pkcs7_message {
    struct x509_certificate *certs;           /* Certificate list */
    struct x509_certificate *crl;            /* Revocation list */
    struct pkcs7_signed_info *signed_infos;
    u8 version;                             /* Version of cert (1 -> PKCS#7 or CMS; 3 -> CMS) */
    bool have_authattrs;                    /* T if have authattrs */

    /* Content Data (or NULL) */
    enum OID data_type;                     /* Type of Data */
    size_t data_len;                        /* Length of Data */
    size_t data_hdrlen;                     /* Length of Data ASN.1 header */
    const void *data;                       /* Content Data (or 0) */
};

```

La structure `pkcs7_message` représente un message signé au format PKCS#7 tel qu'il est manipulé dans le noyau Linux. Elle regroupe l'ensemble des éléments nécessaires à la vérification d'une signature numérique : une liste de certificats X.509 (`certs`) permettant d'identifier les clés publiques des signataires, une liste de certificats révoqués (`crl`), ainsi qu'une liste de structures `pkcs7_signed_info` (`signed_infos`) décrivant les différentes signatures présentes dans le message.

La structure contient également plusieurs informations de contexte, comme la version du format PKCS#7 utilisée ainsi que la présence éventuelle d'attributs authentifiés (`have_authattrs`), qui influencent le mécanisme de vérification de la signature. Enfin, elle encapsule les données signées elles-mêmes (`data`), leur type (`data_type`), représenté par un identifiant OID) ainsi que leur taille.

Il convient désormais d'étudier la structure `struct pkcs7_signed_info` :

```

struct pkcs7_signed_info {
    struct pkcs7_signed_info *next;
    struct x509_certificate *signer; /* Signing certificate (in msg->certs) */
    unsigned index;
};

```

```

    bool                unsupported_crypto;        /* T if not usable due to missing
    ↪ crypto */
    bool                blacklisted;

    /* Message digest - the digest of the Content Data (or NULL) */
    const void          *msgdigest;
    unsigned            msgdigest_len;

    /* Authenticated Attribute data (or NULL) */
    unsigned            authattrs_len;
    const void          *authattrs;
    unsigned long        aa_set;
    ...
    time64_t            signing_time;

    /* Message signature.
    *
    * This contains the generated digest of _either_ the Content Data or
    * the Authenticated Attributes [RFC2315 9.3]. If the latter, one of
    * the attributes contains the digest of the Content Data within it.
    *
    * This also contains the issuing cert serial number and issuer's name
    * [PKCS#7 or CMS ver 1] or issuing cert's SKID [CMS ver 3].
    */
    struct public_key_signature *sig;
};

```

La structure `pkcs7_signed_info` représente une signature individuelle contenue dans un message PKCS#7. Un même message peut en effet contenir plusieurs signatures, chaînées entre elles via le champ `next`.

Chaque entrée associe une signature à son certificat signataire (**signer**) et contient plusieurs informations d'état, telles que la présence d'un algorithme cryptographique non supporté ou le fait que la signature soit révoquée (**blacklisted**).

Elle contient également le digest du message (**msgdigest**), qui correspond soit au hash direct des données signées (cas des modules du noyau), soit au hash des attributs authentifiés (cas du firmware). Dans ce second cas, les attributs authentifiés contiennent eux-mêmes un champ **messageDigest**, qui représente le hash des données originales.

Enfin, le champ central **sig** regroupe les informations cryptographiques nécessaires à la vérification de la signature :

```

struct public_key_signature sig = {
    .s_size              = params->in2_len,
    .digest_size         = params->in_len,
    .encoding            = params->encoding,
    .hash_algo           = params->hash_algo,
    .digest              = (void *)in,
    .s                   = (void *)in2,
};

```

Cette structure regroupe l'ensemble des informations nécessaires à la vérification d'une signature cryptographique dans le noyau Linux. Elle contient le hash du message (**digest**), la signature elle-même (**s**), ainsi que les paramètres associés tels que l'algorithme de hachage, le type d'encodage et les tailles correspondantes. L'objectif est de fournir un objet complet permettant au noyau de vérifier qu'une signature correspond bien au message en utilisant les primitives cryptographiques appropriées.

La fonction `verify_pkcs7_signature()` constitue le point d'entrée principal de ce mécanisme de vérification. Elle prend en entrée les données ainsi qu'un message PKCS#7 contenant la signature, puis enchaîne plusieurs étapes : le parsing du message PKCS#7, la vérification cryptographique de la signature, ainsi que la validation de la chaîne de confiance à l'aide des clés présentes dans les *keyrings* du noyau.

Nous allons maintenant analyser plus en détail la fonction `verify_pkcs7_signature()`, située dans le fichier `linux/certs/systemkeyring.c` :

```

int verify_pkcs7_signature(const void *data, size_t len,
                          const void *raw_pkcs7, size_t pkcs7_len,
                          struct key *trusted_keys,
                          enum key_being_used_for usage,
                          int (*view_content)(void *ctx,
                                              const void *data, size_t len,
                                              size_t asn1hdrlen),
                          void *ctx)
{
    struct pkcs7_message *pkcs7;
    int ret;

    pkcs7 = pkcs7_parse_message(raw_pkcs7, pkcs7_len);
    if (IS_ERR(pkcs7))
        return PTR_ERR(pkcs7);

    ret = verify_pkcs7_message_sig(data, len, pkcs7, trusted_keys, usage,
                                   view_content, ctx);

    pkcs7_free_message(pkcs7);
    pr_devel("<==%s() = %d\n", __func__, ret);
    return ret;
}

```

La fonction `pkcs7_parse_message()` analyse un message PKCS#7 encodé en ASN.1 et construit une structure interne de type `struct pkcs7_message`. Cette étape permet d'extraire les certificats, les signatures ainsi que les données associées, afin de préparer leur vérification par les fonctions du noyau.

Ensuite, la fonction `verify_pkcs7_message_sig()` est appelée afin de valider une signature PKCS#7 sur les données. Cette fonction se trouve dans le fichier `certs/system_keyring.c`. La vérification s'effectue en plusieurs étapes :

```

int verify_pkcs7_message_sig(const void *data, size_t len,
                             struct pkcs7_message *pkcs7,
                             struct key *trusted_keys,
                             enum key_being_used_for usage,
                             int (*view_content)(void *ctx,
                                                  const void *data, size_t len,
                                                  size_t asn1hdrlen),
                             void *ctx)
{
    int ret;

    /* The data should be detached - so we need to supply it. */
    if (data && pkcs7_supply_detached_data(pkcs7, data, len) < 0) {
        pr_err("PKCS#7 signature with non-detached data\n");
        ret = -EBADMSG;
        goto error;
    }

    ret = pkcs7_verify(pkcs7, usage);
    if (ret < 0)
        goto error;

    ret = is_key_on_revocation_list(pkcs7);
    if (ret != -ENOKEY) {
        pr_devel("PKCS#7 key is on revocation list\n");
        goto error;
    }

    if (!trusted_keys) {

```

```

        trusted_keys = builtin_trusted_keys;
    } else if (trusted_keys == VERIFY_USE_SECONDARY_KEYRING) {
#ifdef CONFIG_SECONDARY_TRUSTED_KEYRING
        trusted_keys = secondary_trusted_keys;
#else
        trusted_keys = builtin_trusted_keys;
#endif
    } else if (trusted_keys == VERIFY_USE_PLATFORM_KEYRING) {
#ifdef CONFIG_INTEGRITY_PLATFORM_KEYRING
        trusted_keys = platform_trusted_keys;
#else
        trusted_keys = NULL;
#endif

        if (!trusted_keys) {
            ret = -ENOKEY;
            pr_devel("PKCS#7 platform keyring is not available\n");
            goto error;
        }
    }
    ret = pkcs7_validate_trust(pkcs7, trusted_keys);
    if (ret < 0) {
        if (ret == -ENOKEY)
            pr_devel("PKCS#7 signature not signed with a trusted key\n");
        goto error;
    }

    if (view_content) {
        size_t asnhdrln;

        ret = pkcs7_get_content_data(pkcs7, &data, &len, &asnhdrln);
        if (ret < 0) {
            if (ret == -ENODATA)
                pr_devel("PKCS#7 message does not contain data\n");
            goto error;
        }

        ret = view_content(ctx, data, len, asnhdrln);
    }

error:
    pr_devel("<==%s() = %d\n", __func__, ret);
    return ret;
}

```

Le message PKCS#7 manipulé dans le noyau peut contenir des données détachées. Dans ce cas, celles-ci ne sont pas incluses directement dans la structure ASN.1 et doivent être fournies explicitement.

1 - pkcs7_supply_detached_data() :

Tout d'abord, la fonction `pkcs7_supply_detached_data` est appelée. Cette fonction permet d'associer les données (`data`) au champ `data` de la structure `pkcs7_message`, lorsque celles-ci ne sont pas incluses directement dans le message PKCS#7.

2 - pkcs7_verify() :

Ensuite, la fonction `pkcs7_verify` est appelée pour la vérification cryptographique de la signature. Cette fonction se trouve dans `linux/crypto/asymmetric_keys/pkcs7verify.c`. On peut également retrouver son code source sur GitHub [7].

L'arbre d'appel de cette fonction peut être représenté comme suit :

```

+-- pkcs7_verify(pkcs7, usage)
|   -> vérifie la cohérence interne d'un message PKCS#7
|
|   +-- pkcs7_verify_one(pkcs7, sinfo)
|   |   -> traite une signature individuelle
|   |
|   |   +-- pkcs7_digest(pkcs7, sinfo)

```

```

| | | -> calcule le hash du message PKCS#7
| | | -> compare ce hash avec le champ du msgdigest
| | | -> peut re-hashier les authattrs si nécessaire
| | |
| | +-- pkcs7_find_key(pkcs7, sinfo)
| | | -> cherche le certificat X.509 correspondant
| | | -> match sur issuer + serialNumber ou SKID
| | |
| | +-- public_key_verify_signature(signer->pub, sinfo->sig)
| | | -> vérifie la signature cryptographique (RSA/ECDSA)
| | |
| | +-- pkcs7_verify_sig_chain(pkcs7, sinfo)
| | | -> valide la chaîne de certificats X.509
| | | -> détecte les boucles et blacklist
| | | -> vérifie chaque certificat avec son parent
| | |
| | +-- public_key_verify_signature(parent->pub, x509->sig)
| | | -> vérifie signature de chaque certificat de la chaîne
| +--
+--

```

La fonction `pkcs7_verify()` commence par vérifier les types de signature en fonction de l'usage. Autrement dit, le noyau ne traite pas toutes les signatures PKCS#7 de la même manière : le comportement dépend du contexte d'utilisation, tel que les modules noyau, le firmware, les images kexec ou d'autres usages spécifiques.

Par exemple, pour un module noyau (`.ko`), le message doit contenir des données brutes (`OID_data`) et ne doit pas inclure d'attributs authentifiés (`authattrs`). À l'inverse, pour le firmware, les données sont également brutes, mais les attributs authentifiés sont obligatoires, car ce type de contenu est plus sensible et nécessite le format CMS complet ainsi qu'un niveau de sécurité renforcé.

Ensuite, la fonction procède à trois vérifications principales :

- Le hash du message correspond à celui attendu.
- La signature cryptographique est valide (RSA, ECDSA, etc.).
- La chaîne de certificats X.509 est cohérente et vérifiable jusqu'à une racine de confiance.

Lorsque la fonction `pkcs7_digest(pkcs7, sinfo)` est appelée, le noyau commence par recalculer le hash du message signé afin de le comparer à celui présent dans la signature.

Si le message contient des attributs authentifiés, le noyau vérifie la présence du champ `messageDigest`. Il compare ensuite ce digest avec celui qu'il vient de calculer : en cas de différence, la signature est considérée comme invalide.

Dans ce cas particulier, le hash n'est pas calculé directement sur le message brut, mais sur les *authenticated attributes*. Il convient de préciser que, pour les modules du noyau, le hachage est effectué directement sur le contenu du message, sans utilisation d'attributs authentifiés.

Ensuite, la fonction `pkcs7_find_key()` est appelée afin de retrouver la clé (certificat X.509) à utiliser pour vérifier la signature PKCS#7. PKCS#7 utilise pour cela le nom de l'émetteur (*issuer*) ainsi que le numéro de série du certificat afin d'effectuer la correspondance.

Si un certificat correspondant est trouvé, il est associé à la signature. Dans le cas contraire, le certificat peut être recherché soit dans le message lui-même, soit dans le *keyring* de confiance du noyau (*trusted keys*).

La fonction `public_key_verify_signature()` est ensuite appelée afin de vérifier la signature cryptographique à l'aide de la clé publique associée. On peut trouver la définition de cette fonction dans `linux/crypto/asymmetric_keys/public_key.c`. Elle sélectionne l'algorithme approprié (RSA, ECDSA, etc.), prépare le contexte cryptographique du noyau et exécute la vérification de la signature.

La fonction `pkcs7_verify_sig_chain()` est ensuite utilisée pour valider la chaîne de certificats X.509 incluse dans le message PKCS#7. Elle garantit que chaque certificat est correctement signé par le suivant jusqu'à atteindre une racine de confiance. Concrètement, `pkcs7_verify_sig_chain()` parcourt la chaîne de certificats à partir du certificat du signataire et vérifie sa cohérence. Elle détecte les certificats blacklistés ainsi que les éventuelles boucles dans la chaîne. Chaque certificat est relié à son émetteur via l'issuer, le numéro de série ou le SKID, puis sa signature est vérifiée cryptographiquement à l'aide de la clé publique du certificat parent. Ce processus se poursuit récursivement jusqu'à atteindre un certificat auto-signé, considéré comme la racine de la chaîne de

confiance.

3 - `is_key_on_revocation_list()` :

Ensuite, la fonction `is_key_on_revocation_list(pkcs7)` est appelée afin de vérifier si la clé du certificat a été révoquée ou blacklistée. Dans ce cas, la signature est immédiatement rejetée.

Par la suite, en fonction de la configuration du noyau, un *keyring* de confiance est sélectionné pour la validation. Le noyau charge, au moment du démarrage, différentes sources de clés X.509 dans ses *keyrings* système. Ces clés proviennent de magasins persistants tels que la base de données UEFI ou la liste des clés du propriétaire de la machine (*Machine Owner Key* – MOK). Elles sont utilisées pour vérifier l'intégrité des modules et des binaires exécutés par le système [8].

Les principaux *keyrings* système sont les suivants :

- `.builtin_trusted_keys` : keyring construit lors du démarrage du noyau. Il contient les clés publiques de confiance intégrées au kernel. Son accès est réservé aux utilisateurs privilégiés (root).
- `.platform` : keyring initialisé au démarrage du système. Il contient les clés fournies par la plateforme (par exemple le firmware UEFI) ainsi que des clés publiques personnalisées. Son accès est également restreint aux privilèges root.
- `.blacklist` : keyring contenant les certificats X.509 révoqués. Toute signature utilisant une clé présente dans cette liste est rejetée, même si cette clé figure dans `.builtin_trusted_keys`.

Par défaut, le noyau utilise le keyring construit lors du démarrage. Toutefois, en fonction des options de configuration activées, ce comportement peut varier.

L'option `CONFIG_SECONDARY_TRUSTED_KEYRING` permet de définir un keyring secondaire dans lequel des clés supplémentaires peuvent être ajoutées. Ces clés doivent être validées soit par une clé déjà intégrée au noyau, soit par une clé déjà présente dans ce keyring, tout en étant absentes de la liste de révocation [9].

Si l'option `CONFIG_INTEGRITY_PLATFORM_KEYRING` est activée, le noyau utilise alors `platform_trusted_keys`. L'étape de validation de la confiance consiste à vérifier si le certificat du signataire est présent dans un keyring de confiance.

4 - `pkcs7_validate_trust()` :

La fonction `pkcs7_validate_trust()`, située dans `linux-6.18.5/crypto/asymmetric_keys/pkcs7_trust.c`, vérifie que la chaîne de certificats X.509 d'un message PKCS#7 mène à une clé présente dans un *keyring* de confiance du noyau Linux. Elle parcourt les certificats du signataire jusqu'à atteindre une autorité reconnue (par exemple `builtin`, `secondary` ou `platform`), puis valide cryptographiquement cette relation afin d'établir la confiance de la signature.

Si la chaîne de certificats se termine par un certificat auto-signé inconnu du noyau, la signature PKCS#7 est rejetée, car aucune racine de confiance valide n'a été trouvée.

Dans cette fonction, la vérification cryptographique repose notamment sur l'appel à `verify_signature()` qui permet de vérifier une signature asymétrique à l'aide d'une clé publique stockée dans une structure `key`. Elle commence par vérifier que la clé fournie est bien de type asymétrique (`key_type_asymmetric`) et qu'elle possède un sous-type valide (RSA, ECDSA, etc.).

Ensuite, elle récupère dynamiquement la fonction de vérification associée à l'algorithme via le sous-type :

```
subtype->verify_signature
```

Puis elle appelle cette fonction pour effectuer la vérification cryptographique réelle :

```
ret = subtype->verify_signature(key, sig);
```

Cette abstraction permet au noyau de supporter plusieurs algorithmes de signature sans modifier le code PKCS#7 ou X.509. La vérification concrète est ainsi déléguée dynamiquement au backend cryptographique approprié.

5 - `pkcs7_get_content_data()` :

La fonction `pkcs7_get_content_data()`, située dans `linux-6.18.5/crypto/asymmetric_keys/pkcs7_parser.c`, permet d'accéder au contenu brut (`data`) stocké dans un message PKCS#7 déjà parsé par le noyau Linux.

Elle retourne un pointeur vers les données, leur taille, ainsi que, de manière optionnelle, la taille de l'en-tête associé. Après la validation de la signature PKCS#7, cette fonction est utilisée pour extraire le message signé. Celui-ci est ensuite transmis à une fonction de traitement (`view_content`) afin d'être exploité par le noyau ou par le sous-système appelant.

2.3 Vérification de la compatibilité architecturale

Avant de charger un module, le noyau commence par vérifier que le module a été compilé pour la même architecture matérielle que celle du système hôte (par exemple `x86_64`, `ARM` ou `MIPS`).

Cette vérification s'appuie sur la validation du format ELF du module. Elle est notamment réalisée par la fonction `elf_validity_cache_copy()` :

```
static int elf_validity_cache_copy(struct load_info *info, int flags)
{
    int err;

    err = elf_validity_cache_sechdrs(info);
    if (err < 0)
        return err;
    err = elf_validity_cache_secstrings(info);
    if (err < 0)
        return err;
    err = elf_validity_cache_index(info, flags);
    if (err < 0)
        return err;
    err = elf_validity_cache_strtab(info);
    if (err < 0)
        return err;
    ...
}
```

Cette fonction effectue plusieurs contrôles sur la structure ELF du module :

- validation des en-têtes de sections ;
- vérification des tables de chaînes ;
- contrôle des index internes ;
- accès aux métadonnées du module.

Ces vérifications garantissent que le module est un objet ELF valide et cohérent avec l'architecture du noyau avant toute allocation mémoire ou exécution de code. Un module compilé pour une architecture différente est automatiquement rejeté.

2.4 Vérification de la version du noyau (`vermagic`)

`vermagic` est une chaîne d'identification magique présente dans le noyau Linux et visible dans `MODULE_INFO` sur les modules du noyau. Il sert à vérifier que le module noyau a été compilé avec une version compatible du noyau ainsi qu'une configuration correspondante :

```
root@vm:~$ modinfo my_module.ko
...
vermagic:      6.10.7 preempt mod_unload
```

Le `vermagic` du noyau est défini en fonction des configurations au moment de compilation sur `include/linux/vermagic.h` :

```
#define VERMAGIC_STRING          \
    UTS_RELEASE " "              \
    MODULE_VERMAGIC_SMP MODULE_VERMAGIC_PREEMPT          \
    MODULE_VERMAGIC_MODULE_UNLOAD MODULE_VERMAGIC_MODVERSIONS \
    MODULE_ARCH_VERMAGIC         \
    MODULE_RANDSTRUCT

#endif
```

Chaque partie de cette chaîne dépend directement des options de configuration activées lors de la compilation du noyau par exemple :

```

/* Simply sanity version stamp for modules. */
#ifdef CONFIG_SMP
#define MODULE_VERMAGIC_SMP "SMP "
#else
#define MODULE_VERMAGIC_SMP ""
#endif
#ifdef CONFIG_PREEMPT_RT
#define MODULE_VERMAGIC_PREEMPT "preempt_rt "
#elif defined(CONFIG_PREEMPT_BUILD)
#define MODULE_VERMAGIC_PREEMPT "preempt "
#else
#define MODULE_VERMAGIC_PREEMPT ""
#endif
#ifdef CONFIG_MODULE_UNLOAD
#define MODULE_VERMAGIC_MODULE_UNLOAD "mod_unload "
...
#ifdef CONFIG_MODVERSIONS
#define MODULE_VERMAGIC_MODVERSIONS "modversions "
...
#ifdef RANDSTRUCT
#define MODULE_RANDSTRUCT "RANDSTRUCT_" RANDSTRUCT_HASHED_SEED
...
#endif

```

Cette chaîne est ensuite intégrée dans chaque module .ko. Elle permet au noyau de vérifier, au moment du chargement, que le module a bien été compilé pour la même version et avec la même configuration. Autrement dit, le noyau s'assure que le module est compatible avec son propre environnement d'exécution.

Voyons les différents éléments constituant la chaîne `VERMAGIC_STRING` :

- **UTS_RELEASE** : correspond à la version exacte du noyau utilisé pour la compilation du module.
- **SMP** : si l'option `CONFIG_SMP` est activée (support multiprocesseur), la chaîne "SMP" est ajoutée.
- **Préemption** :
 - `preempt_rt` : noyau temps réel (`CONFIG_PREEMPT_RT`).
 - `preempt` : noyau préemptif standard (`CONFIG_PREEMPT_BUILD`).
 - aucune mention : noyau non préemptif.
- **Module unload** : si `CONFIG_MODULE_UNLOAD` est activé, la chaîne "mod_unload" est ajoutée. Cela indique que le noyau autorise le déchargement des modules.
- **Modversions** : si `CONFIG_MODVERSIONS` est activé, la chaîne "modversions" est ajoutée. Cette option permet une vérification fine de la compatibilité des symboles exportés.
- **Architecture** : dépend de l'architecture cible (ex. `x86_64`, `arm64`, etc.) et est définie dans `asm/vermagic.h`.
- **Randstruct** : si la protection `RANDSTRUCT` est activée, une graine unique est ajoutée à la chaîne. Cela empêche le chargement d'un module compilé avec une disposition mémoire des structures différente de celle du noyau.

Le noyau compare la version avec sa propre version et ses options critiques. Toute divergence conduit à une erreur de type `invalid module format`.

```

root@vm:~$ insmod my_module.ko
insmod: can't insert 'my_module.ko': invalid module format
root@vm:~$ dmesg | tail -n 1
my_module: version magic '6.10.7 preempt mod_unload ' should be '6.10.7 preempt mod_unload
↳ modversions'

```

Dans l'exemple ci-dessus, l'option `modversions` est activée. Si cette option est activée côté noyau mais absente côté module (ou inversement), le chargement échoue.

La fonction `check_modinfo()` réalise une vérification ciblée du champ `vermagic` du module avant toute initialisation. Son rôle est de s'assurer que le module est compatible avec le noyau courant et ses options critiques.

```

static int check_modinfo(struct module *mod, struct load_info *info, int flags)
{
    const char *modmagic = get_modinfo(info, "vermagic");
    int err;

    if (flags & MODULE_INIT_IGNORE_VERMAGIC)
        modmagic = NULL;

    /* This is allowed: modprobe --force will invalidate it. */
    if (!modmagic) {
        err = try_to_force_load(mod, "bad vermagic");
        if (err)
            return err;
    } else if (!same_magic(modmagic, vermagic, info->index.vers)) {
        pr_err("%s: version magic '%s' should be '%s'\n",
            info->name, modmagic, vermagic);
        return -ENOEXEC;
    }

    err = check_modinfo_livepatch(mod, info);
    if (err)
        return err;

    return 0;
}

```

La chaîne `vermagic` du module est récupérée via `get_modinfo()`. Si l'option `MODULE_INIT_IGNORE_VERMAGIC` est activée ou si aucun `vermagic` n'est présent, le noyau tente de charger le module malgré tout avec `try_to_force_load()`. (chargement forcé, lorsque l'utilisateur a explicitement demandé cette option). Sinon si cette chaîne est présent mais ne correspond pas à la valeur attendue, un message d'erreur est généré et le chargement échoue.

Un point intéressant est que lorsque l'option `CONFIG_MODVERSIONS` est activée, le noyau ne compare plus la chaîne `vermagic` :

```

/* First part is kernel version, which we ignore if module has crcs. */
int same_magic(const char *amagic, const char *bmagic,
               bool has_crcs)
{
    if (has_crcs) {
        amagic += strcspn(amagic, " ");
        bmagic += strcspn(bmagic, " ");
    }
    return strcmp(amagic, bmagic) == 0;
}

```

Il s'appuie essentiellement sur les CRC des symboles importés, stockés dans la section `__versions`. Cela signifie que deux noyaux de versions différentes peuvent accepter le même module tant que les CRC de tous les symboles requis correspondent. Comme illustré dans la preuve de concept 4.3.2, le noyau vérifie les CRC des symboles et ignore la chaîne `vermagic`.

2.5 Vérification de la structure `struct module`

Lors du chargement d'un module noyau (`.ko`), le noyau Linux vérifie la cohérence de la structure interne `struct module`. Cette structure est cruciale, car elle représente le module du point de vue du noyau et contient :

- l'état du module (chargé, initialisé, déchargé) ;
- les symboles exportés et leur version ;
- l'intégration dans `sysfs` ;
- la gestion des paramètres et des options de compilation ;

- les informations de version et de dépendances.

Le code ci-dessous illustre la définition de `struct module` pour quelques-uns de ces champs :

```
struct module {
    enum module_state state;

    /* Member of list of modules */
    struct list_head list;

    /* Unique handle for this module */
    char name[MODULE_NAME_LEN];

#ifdef CONFIG_STACKTRACE_BUILD_ID
    /* Module build ID */
    unsigned char build_id[BUILD_ID_SIZE_MAX];
#endif

    /* Sysfs stuff. */
    struct module_kobject mkobj;
    struct module_attribute *modinfo_attrs;
    const char *version;
    const char *srcversion;
    struct kobject *holders_dir;

    /* Exported symbols */
    const struct kernel_symbol *syms;
    const u32 *crcls;
    unsigned int num_syms;

#ifdef CONFIG_ARCH_USES_CFI_TRAPS
    ...
#endif
    /* Kernel parameters. */
#ifdef CONFIG_SYSFS
    ...
#endif
#ifdef CONFIG_MODULE_SIG
    ...
#endif
    ...
};
```

Cependant, la définition de `struct module` n'est pas fixe. Elle dépend de la configuration du noyau (`.config`) et des directives de compilation conditionnelle (`#ifdef CONFIG_*`).

Selon les options activées ou désactivées, comme `CONFIG_MODVERSIONS`, `CONFIG_SYSFS`, `CONFIG_STACKTRACE_BUILD_ID` ou `CONFIG_KALLSYMS`, la taille, l'ordre et la présence des champs dans `struct module` peuvent varier. Cette variabilité explique pourquoi le noyau doit vérifier la cohérence de la structure avant tout chargement.

Il existe ainsi deux niveaux de protection :

- **Une vérification structurelle minimale**, toujours active, consistant notamment à comparer la taille de la section `.gnu.linkonce.this_module` avec `sizeof(struct module)` du noyau en cours d'exécution ;
- **Une vérification ABI fine par CRC**, activée uniquement lorsque `CONFIG_MODVERSIONS=y`, reposant sur la comparaison des signatures (CRC) des symboles exportés, notamment `module_layout`. Ce second mécanisme fera l'objet d'une analyse détaillée dans la section suivante.

Dans un premier temps, nous nous concentrons sur le premier niveau, à savoir la vérification structurelle minimale.

Lors de la compilation d'un module, une instance binaire de `struct module` est générée et intégrée dans la section ELF spéciale `.gnu.linkonce.this_module` du fichier `.ko`.

Au moment de l'insertion du module (`insmod`), le noyau compare cette instance avec sa propre définition de `struct module`, construite à partir de la configuration du noyau en cours d'exécution.

Cette vérification se fait par la fonction `elf_validity_cache_index_mod()` :

```
static int elf_validity_cache_index_mod(struct load_info *info)
{
    Elf_Shdr *shdr;
    int mod_idx;

    mod_idx = find_any_unique_sec(info, ".gnu.linkonce.this_module");
    if (mod_idx <= 0) {
        pr_err("module %s: Exactly one .gnu.linkonce.this_module section must
        ↪ exist.\n",
            info->name ? "(missing .modinfo section or name field)");
        return -ENOEXEC;
    }

    shdr = &info->sechdrs[mod_idx];

    if (shdr->sh_type == SHT_NOBITS) {
        pr_err("module %s: .gnu.linkonce.this_module section must have a size set\n",
            info->name ? "(missing .modinfo section or name field)");
        return -ENOEXEC;
    }

    if (!(shdr->sh_flags & SHF_ALLOC)) {
        pr_err("module %s: .gnu.linkonce.this_module must occupy memory during process
        ↪ execution\n",
            info->name ? "(missing .modinfo section or name field)");
        return -ENOEXEC;
    }

    if (shdr->sh_size != sizeof(struct module)) {
        pr_err("module %s: .gnu.linkonce.this_module section size must match the
        ↪ kernel's built struct module size at run time\n",
            info->name ? "(missing .modinfo section or name field)");
        return -ENOEXEC;
    }

    info->index.mod = mod_idx;

    return 0;
}
```

La fonction `elf_validity_cache_index_mod()` valide la section ELF spéciale `.gnu.linkonce.this_module`, générée automatiquement lors de la compilation du module.

Cette section contient l'instance binaire de `struct module` (`__this_module`), qui sera utilisée par le noyau pour représenter le module chargé.

Plusieurs vérifications sont effectuées :

- la section doit exister et être unique ;
- elle ne doit pas être de type `SHT_NOBITS` (elle doit contenir des données réelles) ;
- elle doit posséder le drapeau `SHF_ALLOC`, garantissant son allocation en mémoire ;
- sa taille doit être strictement égale à `sizeof(struct module)` du noyau en cours d'exécution.

Si une différence de taille ou de disposition mémoire (layout) est détectée, le chargement du module échoue avec une erreur de type `invalid module format`. Cette vérification empêche toute incohérence structurelle pouvant conduire à une corruption mémoire du noyau.

```

root@vm:~$ insmod my_module.ko
module module: .gnu.linkonce.this_module section size must match the kernel's built struct
↳ module size at run time
insmod: can't insert 'my_module.ko': invalid module format

```

L'instance de `struct module` intégrée dans un module noyau est stockée dans la section ELF `.gnu.linkonce.this_module`, accessible via des outils tels que `readelf` ou `objdump`.

```

toto@LAPTOP-T02FKT7J:/vm/mnt$ readelf -S my_module.ko | grep gnu.linkonce
[22] .gnu.linkonce.thi PROGBITS          0000000000000000 000001c0

```

Le symbole `__this_module` référence directement cette structure :

```

toto@LAPTOP-T02FKT7J:/vm/mnt$ nm my_module.ko | grep __this_module
0000000000000000 D __this_module

```

L'analyse du layout réel de `struct module`, incluant les offsets et tailles des champs, peut être réalisée à l'aide de l'outil `pahole` sur le binaire `vmlinux`, reflétant précisément la configuration du noyau en cours d'exécution.

```

toto@LAPTOP-T02FKT7J:/vm/mnt$ pahole -C module vmlinux
struct module {
    enum module_state      state;           /*    0    4 */
    /* XXX 4 bytes hole, try to pack */
    struct list_head       list;           /*    8   16 */
    ...

    struct list_head       source_list;     /*  960   16 */
    struct list_head       target_list;     /*  976   16 */
    void                   (*exit)(void);   /*  992    8 */
    atomic_t               refcnt;          /* 1000    4 */
    /* size: 1024, cachelines: 16, members: 44 */
    /* sum members: 982, holes: 5, sum holes: 22 */
    /* padding: 20 */
    /* forced alignments: 1, forced holes: 1, sum forced holes: 8 */
} __attribute__((__aligned__(64)));

```

Les informations présentées par l'outil `pahole` indiquent la disposition réelle de chaque membre en mémoire, par rapport au début de la structure. Chaque membre est accompagné de son **offset** et de sa taille. Les **trous** (holes) identifiés correspondent à des octets de **padding** insérés par le compilateur afin de respecter les contraintes d'**alignement** et optimiser les performances. Ici la taille totale de `struct module` est de 1024 octets, répartis sur exactement 16 cachelines.

Pour le module analysé précédemment, la taille de `struct module` diffère :

```

toto@LAPTOP-T02FKT7J:/vm/mnt$ pahole -C module my_module.ko
struct module {
    enum module_state      state;           /*    0    4 */
    /* XXX 4 bytes hole, try to pack */
    struct list_head       list;           /*    8   16 */
    ...

    /* size: 704, cachelines: 11, members: 47 */
    /* sum members: 650, holes: 6, sum holes: 26 */
    /* padding: 28 */
    /* forced alignments: 2, forced holes: 1, sum forced holes: 8 */
} __attribute__((__aligned__(64)));

```

Le décalage de taille entre le noyau (1024 octets) et le module (704 octets) provoque une incompatibilité, car les champs de la structure du module ne sont plus alignés conformément aux attentes du noyau. Par conséquent, une erreur "invalid module format" est générée.

2.6 Vérification des versions de symboles (CRC)

2.6.1 Mécanisme de vérification des versions de symboles

Comme introduit précédemment, lorsque CONFIG_MODVERSIONS=y, le noyau met en œuvre un mécanisme de vérification ABI (Application Binary Interface) basé sur des CRC.(Cyclic Redundancy Check).

Chaque symbole exporté par le noyau est associé à un CRC calculé lors de la compilation à partir de son interface : prototype, types utilisés et structures dépendantes.

Lors de la compilation d'un module, les CRC des symboles utilisés sont embarqués dans la section ELF __versions. Au moment du chargement, le noyau compare ces CRC avec ceux présents dans le noyau en cours d'exécution.

Cette vérification est réalisée par la fonction check_version() :

```
int check_version(const struct load_info *info,
                  const char *symname,
                  struct module *mod,
                  const u32 *crc)
{
    Elf_Shdr *sechdrs = info->sechdrs;
    unsigned int versindex = info->index.vers;
    unsigned int i, num_versions;
    struct modversion_info *versions;
    struct modversion_info_ext version_ext;

    /* Exporting module didn't supply crcs? OK, we're already tainted. */
    if (!crc)
        return 1;

    /* If we have extended version info, rely on it */
    if (info->index.vers_ext_crc) {
        for_each_modversion_info_ext(version_ext, info) {
            if (strcmp(version_ext.name, symname) != 0)
                continue;
            if (*version_ext.crc == *crc)
                return 1;
            pr_debug("Found checksum %X vs module %X\n",
                    *crc, *version_ext.crc);
            goto bad_version;
        }
        pr_warn_once("%s: no extended symbol version for %s\n",
                    info->name, symname);
        return 1;
    }

    /* No versions at all? modprobe --force does this. */
    if (versindex == 0)
        return try_to_force_load(mod, symname) == 0;

    versions = (void *)sechdrs[versindex].sh_addr;
    num_versions = sechdrs[versindex].sh_size
        / sizeof(struct modversion_info);

    for (i = 0; i < num_versions; i++) {
        u32 crcval;

        if (strcmp(versions[i].name, symname) != 0)
            continue;

        crcval = *crc;
        if (versions[i].crc == crcval)
            return 1;
        pr_debug("Found checksum %X vs module %lX\n",
                crcval, versions[i].crc);
        goto bad_version;
    }
}
```

```

    }

    /* Broken toolchain. Warn once, then let it go.. */
    pr_warn_once("%s: no symbol version for %s\n", info->name, symname);
    return 1;

bad_version:
    pr_warn("%s: disagrees about version of symbol %s\n", info->name, symname);
    return 0;
}

```

La fonction parcourt la table des versions du module et compare, pour chaque symbole, le CRC attendu avec celui fourni par le noyau. Si les CRC correspondent, le symbole est considéré comme compatible. En revanche, une divergence indique une incompatibilité ABI et entraîne l'échec du chargement afin d'éviter toute corruption mémoire ou comportement indéfini.

Plusieurs cas particuliers sont également gérés :

- absence de CRC côté noyau ou module ;
- absence de section `__versions` (par exemple lors d'un chargement forcé) ;
- incohérences liées à la toolchain.

Dans certaines situations, le noyau peut autoriser le chargement via `try_to_force_load()`, tout en marquant le noyau comme *tainted*.

La fonction `check_export_symbol_versions()` complète ce mécanisme en s'assurant que les modules exportant eux-mêmes des symboles fournissent leurs CRC lorsque `CONFIG_MODVERSIONS` est activé :

```

static int check_export_symbol_versions(struct module *mod) {
    #ifdef CONFIG_MODVERSIONS
        if ((mod->num_syms && !mod->crcls) || (mod->num_gpl_syms && !mod->gpl_crcls)) {
            return try_to_force_load(mod, "no versions for exported symbols");
        }
    #endif
    return 0;
}

```

Cas particulier de `struct module` :

La compatibilité de `struct module` est vérifiée indirectement via le symbole exporté `module_layout`. Cette vérification est effectuée par la fonction `check_modstruct_version()` :

```

int check_modstruct_version(const struct load_info *info,
                           struct module *mod)
{
    struct find_symbol_arg fsa = {
        .name      = "module_layout",
        .gplok     = true,
    };
    bool have_symbol;

    /*
     * Since this should be found in kernel (which can't be removed), no
     * locking is necessary. Regardless use a RCU read section to keep
     * lockdep happy.
     */
    scoped_guard(rcu)
        have_symbol = find_symbol(&fsa);
    BUG_ON(!have_symbol);

    return check_version(info, "module_layout", mod, fsa.crc);
}

```


Cette fonction recherche le symbole exporté `module_layout` dans le noyau, récupère son CRC, puis appelle `check_version()` afin de comparer ce CRC avec celui embarqué dans le module lorsque `CONFIG_MODVERSIONS=y`;

Le symbole `module_layout` est défini dans le noyau comme suit :

```
/*
 * Generate the signature for all relevant module structures here.
 * If these change, we don't want to try to parse the module.
 */
void module_layout(struct module *mod,
                   struct modversion_info *ver,
                   struct kernel_param *kp,
                   struct kernel_symbol *ks,
                   struct tracepoint * const *tp)
```

Ce symbole dépend de plusieurs structures internes critiques, notamment `struct module`, `struct modversion_info` et `struct kernel_param`. Toute modification de ces structures entraîne une modification du CRC associé à `module_layout`.

Lors du chargement, le noyau compare :

- le CRC embarqué dans la section `__versions` du module;
- le CRC du symbole correspondant dans le noyau.

Si les CRC diffèrent, la fonction `check_version()` retourne une erreur, ce qui provoque l'échec du chargement avec le message `invalid module format`.

2.6.2 Infrastructure de versionnement par CRC des symboles

Lors de la compilation du noyau Linux, un fichier nommé `Module.symvers` est généré. Ce fichier contient l'ensemble des symboles exportés par le noyau ainsi que par les modules compilés. Pour chaque symbole exporté, une valeur de CRC associée est également stockée.

Le format du fichier `Module.symvers` est le suivant [10] :

<CRC>	<Symbole>	<Module>	<Type d'export>	<Namespace>
0xe1cc2a05	usb_stor_suspend	drivers/usb/storage/usb-storage	EXPORT_SYMBOL_GPL	USB_STORAGE

Les champs sont séparés par des tabulations et peuvent être vides, par exemple lorsqu'aucun namespace n'est associé à un symbole exporté. Lorsque l'option `CONFIG_MODVERSIONS` n'est pas activée, la valeur du CRC est fixée à `0x00000000`.

Comme illustré dans la preuve de concept 4.3, lors de la compilation d'un module pour un noyau donné, les CRC stockés dans le champ `__versions` du module correspondent à ceux présents dans le fichier `/lib/modules/<version>/build/Module.symvers`.

Ainsi, Le fichier `Module.symvers` remplit deux rôles principaux :

- il liste l'ensemble des symboles exportés par `vmLinux` et les modules du noyau;
- il stocke les CRC associés aux symboles lorsque `CONFIG_MODVERSIONS` est activé.

Avec l'option `CONFIG_MODVERSIONS` activée, le processus de compilation du noyau devient plus complexe. Lorsqu'un module utilise `EXPORT_SYMBOL`, l'outil `genksyms` est invoqué. Pour chaque symbole exporté, un CRC correspondant à la signature du symbole est calculé. Une section dédiée est ensuite créée (`__kcrctab_<fonction>`), contenant une valeur de type `long` représentant ce CRC.

Ces sections sont ensuite regroupées par le script de linker `scripts/module-common.lds` pour former la section finale `__kcrctab`. Un mécanisme similaire s'applique aux variantes `EXPORT_SYMBOL_GPL`, `EXPORT_SYMBOL_GPL_FUTURE` et `EXPORT_SYMBOL_NS`, mais avec des préfixes différents.

Du côté des symboles utilisés (imports), le CRC est recherché dans le fichier `Module.symvers`, généré lors de la compilation du noyau ou des modules lorsque `CONFIG_MODVERSIONS` est activé.

Enfin, le nom du symbole et son CRC sont stockés dans le tableau `__versions` du module importateur, dans la section ELF correspondante [11].

Voyons trois configurations liées à la gestion des CRC dans le mécanisme de vérification des versions de symboles :

- Lorsque l’option `CONFIG_MODVERSIONS` est activée, les CRC associés aux symboles exportés sont également enregistrés dans les sections `__kcrctab` ou `__kcrctab_gpl`.
- Lorsque l’option `CONFIG_BASIC_MODVERSIONS` est activée (option par défaut avec `CONFIG_MODVERSIONS`), les symboles importés par les modules contiennent leur nom ainsi que leur CRC dans la section `__versions` du module importateur. Ce mode impose toutefois une limitation sur la taille des noms de symboles, qui ne doivent pas dépasser 64 octets.
- Enfin, lorsque l’option `CONFIG_EXTENDED_MODVERSIONS` est activée (requis notamment pour utiliser simultanément `CONFIG_MODVERSIONS` et `CONFIG_RUST`), les symboles importés sont traités différemment : leurs noms sont stockés dans la section `__version_ext_names` sous forme de chaînes concaténées et null-terminées, tandis que les CRC correspondants sont stockés dans la section `__version_ext_crcs`.

Lors de la compilation d’un module externe, le système de build doit accéder aux symboles du noyau afin de vérifier que tous les symboles externes sont correctement définis. Cette vérification est effectuée lors de l’étape `MODPOST`.

À cette étape, `modpost` récupère les symboles en lisant le fichier `Module.symvers` présent dans l’arborescence des sources du noyau. Au cours de cette phase, un nouveau fichier `Module.symvers` est également généré, contenant l’ensemble des symboles exportés par le module externe.

2.6.3 Génération des CRC avec `genksyms`

L’outil `genksyms` est utilisé lors de la compilation du noyau Linux afin de générer des informations de version associées aux symboles exportés. Il prend en entrée du code source C prétraité (généralement produit par la commande `gcc -E`) et produit des empreintes de type CRC basées sur la structure des symboles.

Le fonctionnement de `genksyms` repose sur plusieurs étapes successives. Dans un premier temps, il analyse les définitions présentes dans le code source, notamment les `typedef`, `struct`, `union` et `enum`. Il identifie également les symboles globaux et enregistre les informations nécessaires pour reconstruire leur définition complète.

Lorsqu’un symbole est traité, `genksyms` reconstruit récursivement sa définition complète en développant toutes ses dépendances (types, structures, etc.). Cette représentation finale est ensuite utilisée comme entrée d’un algorithme de CRC, qui produit une valeur entière. Cette valeur change dès qu’une des définitions incluses est modifiée pour ce symbole [12].

Le code source de `genksyms` se trouve dans le fichier `scripts/genksyms/genksyms.c` du noyau Linux. Il est également possible de consulter ce code en ligne, notamment sur GitHub [13].

Le traitement du code source repose sur une chaîne classique d’analyse lexicale et syntaxique.

Tout d’abord, le code source C est lu par un analyseur lexical (*lexer*, généré par `flex`), qui transforme le flux de caractères en une suite de *tokens*.

Ces tokens sont ensuite transmis à l’analyseur syntaxique (*parser*, généré par `bison`), dont la fonction principale est `yyparse()`. Le parser reconnaît les structures du langage C (déclarations, types, etc.) et construit une représentation interne des symboles.

Contrairement à un compilateur classique, `genksyms` ne construit pas un arbre de syntaxe abstraite (AST) complet. Il utilise à la place une représentation plus légère, basée sur des listes chaînées et une table de symboles.

Les définitions des symboles sont représentées sous forme de listes chaînées de type `string_list`, où chaque nœud correspond à un fragment de code :

```

struct string_list {
    struct string_list *next;
    enum symbol_type tag;
    char *string;
};

```

Cette structure représente une séquence linéaire de fragments du code source. Chaque nœud contient une chaîne de caractères ainsi qu'un type (**tag**) indiquant la nature du fragment. Les symboles extraits du code sont stockés dans une table de hachage à l'aide de la structure suivante :

```

struct symbol {
    struct hlist_node hnode;
    char *name;
    enum symbol_type type;
    struct string_list *defn;
    struct symbol *expansion_trail;
    struct symbol *visited;
    int is_extern;
    int is_declared;
    enum symbol_status status;
    int is_override;
};

```

Le type du symbole est défini par :

```

enum symbol_type {
    SYM_NORMAL,
    SYM_TYPEDEF,
    SYM_ENUM,
    SYM_STRUCT,
    SYM_UNION,
    SYM_ENUM_CONST
};

```

Chaque entrée représente un symbole du programme (fonction, variable, type, etc.).

- Le champ **name** contient le nom du symbole, tandis que **type** indique sa catégorie (par exemple : SYM_NORMAL, SYM_STRUCT, SYM_TYPEDEF, etc.).
- Le champ **defn** est particulièrement important car il contient la définition du symbole sous forme de liste chaînée (**string_list**). Cette représentation permet de capturer la structure textuelle du symbole sans construire un AST complet.
- Le champ **hnode** permet d'insérer le symbole dans une table de hachage, offrant un accès rapide lors des opérations de recherche (via **find_symbol**).
- Le champ **expansion_trail** est utilisé pour suivre dynamiquement la chaîne d'expansion en cours. Lorsqu'un symbole est en cours de traitement, ce champ est temporairement utilisé pour représenter les symboles actuellement en cours d'expansion.
- Le champ **visited**, est utilisé après l'expansion pour mémoriser les symboles qui ont été parcourus. Il permet de construire une liste chaînée de tous les symboles visités lors du calcul du CRC. Contrairement à **expansion_trail**, qui est temporaire et lié à la pile d'exécution, **visited** sert à conserver une trace persistante des symboles traités.

- Les champs `is_extern`, `is_declared`, `status` et `is_override` permettent de gérer l'état et l'évolution des symboles au cours de l'analyse.

Lorsqu'un symbole est reconnu par le parser, sa définition est construite sous forme d'une liste chaînée stockée dans le champ `defn`.

Une fois cette représentation construite, le symbole est inséré dans table de hachage via la fonction `add_symbol()`. Cette fonction assure la cohérence globale des symboles en détectant les éventuelles redéfinitions, en comparant la nouvelle définition avec une définition existante, et en mettant à jour le symbole si nécessaire ou en créant un nouveau.

La fonction `find_symbol()` permet ensuite de rechercher un symbole à partir de son nom. Elle calcule un hash du nom à l'aide de la fonction `crc32(name)`, accède au compartiment correspondant dans la table de hachage, puis vérifie l'égalité du nom ainsi que la compatibilité du type.

Une fois les symboles correctement enregistrés et accessibles, `gensyms` calcule une empreinte représentative de leur définition à l'aide d'un algorithme de hachage de type CRC32.

Le CRC32(Cyclic Redundancy Check sur 32 bits) est un algorithme de somme de contrôle conçu pour détecter les modifications accidentelles des données. Il produit une valeur de 32 bits, généralement représentée sous forme d'une chaîne hexadécimale de 8 caractères, qui résume le contenu de l'entrée.

Ses principales caractéristiques sont les suivantes :

- **Déterministe** : une même entrée produit toujours la même empreinte ;
- **Très sensible** : la modification d'un seul caractère entraîne un CRC complètement différent ;
- **Rapide** : le calcul est léger et efficace ;
- **Longueur fixe** : le résultat est toujours une valeur de 32 bits (8 caractères hexadécimaux).

Pour plus de détails sur l'algorithme, le lecteur peut consulter [14, 15].

Dans l'implémentation de `gensyms`, le calcul du CRC repose sur une table statique `crctab32`, qui constitue une table de pré-calcul (*lookup table*). Cette optimisation permet d'éviter des calculs bit à bit coûteux, en remplaçant ces opérations par de simples accès mémoire, ce qui améliore significativement les performances.

Pour traiter un octet, la fonction `partial_crc32_one` est utilisée :

```
static uint32_t partial_crc32_one(uint8_t c, uint32_t crc)
{
    return crctab32[(crc ^ c) & 0xff] ^ (crc >> 8);
}
```

Cette fonction met à jour la valeur du CRC en intégrant un nouvel octet `c`. L'opération `crc ^ c` (XOR) combine les bits du caractère avec ceux du CRC courant, ce qui permet d'incorporer l'information du nouvel octet dans le calcul.

L'expression `(crc ^ c) & 0xff` permet ensuite d'extraire les 8 bits de poids faible, qui servent d'indice pour accéder à la table pré-calculée `crctab32`. Cette table contient des valeurs permettant d'accélérer le calcul du CRC en évitant des opérations bit à bit coûteuses. Par ailleurs, l'instruction `crc >> 8` effectue un décalage du CRC courant de 8 bits vers la droite. Cela correspond à l'élimination des bits déjà traités, afin de faire de la place pour l'intégration du nouvel octet. Enfin, les deux résultats sont combinés par une opération XOR, produisant ainsi la nouvelle valeur du CRC.

Pour traiter une chaîne de caractères complète, la fonction suivante est utilisée :

```
static uint32_t partial_crc32(const char *s, uint32_t crc)
{
    while (*s)
        crc = partial_crc32_one(*s++, crc);
    return crc;
}
```

La fonction `partial_crc32` parcourt la chaîne caractère par caractère et met à jour progressivement la valeur du CRC en appelant `partial_crc32_one` pour chaque octet.

LL'expression `*s++` signifie que le caractère pointé par `s` est utilisé, puis que le pointeur est avancé. Ainsi, la chaîne est parcourue séquentiellement de gauche à droite, du premier caractère jusqu'au dernier.

Le calcul du CRC est **incrémental** et à chaque itération, la nouvelle valeur du CRC dépend à la fois du caractère courant et de la valeur précédente du CRC. Ainsi, le résultat obtenu après le traitement d'un caractère est réutilisé pour le caractère suivant, ce qui permet de propager l'information sur toute la chaîne.

Enfin, la fonction suivante fournit une interface complète pour le calcul du CRC32 :

```
static uint32_t crc32(const char *s)
{
    return partial_crc32(s, 0xffffffff) ^ 0xffffffff;
}
```

La fonction `crc32` initialise le CRC avec la valeur `0xffffffff`, applique le calcul sur l'ensemble de la chaîne via `partial_crc32`, puis effectue une inversion finale des bits (opération XOR avec `0xffffffff`).

Ainsi, l'une des fonctions centrales de `gensyms` est `expand_and_crc_sym()`.

La fonction `expand_and_crc_sym` calcule un hash (CRC) représentant un symbole en explorant récursivement sa définition ainsi que ses dépendances. Chaque élément de la définition (stockée sous forme de `string_list`) est traité en fonction de son type :

- **SYM_NORMAL** : la chaîne est directement intégrée dans le calcul du CRC via `partial_crc32`, suivie de l'ajout d'un espace (' ') comme séparateur.
- **SYM_TYPEDEF / SYM_ENUM_CONST** : le symbole référencé est recherché via `find_symbol()`. Si ce symbole est déjà en cours d'expansion (indiqué par le champ `expansion_trail`), seule son identité (nom) est prise en compte afin d'éviter une récursion infinie. Dans le cas contraire, la fonction effectue une expansion récursive complète de sa définition.
- **SYM_STRUCT / SYM_UNION / SYM_ENUM** : si le symbole n'est pas défini, un symbole fictif contenant `UNKNOWN` est créé afin de signaler une définition manquante tout en poursuivant le calcul.
Si le symbole est déjà en cours d'expansion, la fonction n'explore pas sa définition en profondeur. Elle se contente d'intégrer dans le CRC le mot-clé (`struct`, `union`, `enum`) suivi du nom du symbole. Dans le cas contraire, une expansion récursive complète de la définition est effectuée.

La fonction `export_symbol()` constitue le point d'entrée pour le calcul du CRC d'un symbole exporté. Elle commence par rechercher le symbole à l'aide de `find_symbol()`. Si celui-ci est trouvé, elle initialise le CRC à `0xffffffff`, puis lance l'expansion complète via `expand_and_crc_sym()`.

Le résultat obtenu est ensuite inversé (opération XOR avec `0xffffffff`) afin de finaliser le calcul du CRC32. Enfin, la valeur est affichée sous la forme suivante :

```
#SYMVER nom_du_symbole 0XXXXXXXXX
```

En parallèle, la fonction parcourt la chaîne `expansion_trail` afin d'identifier les symboles dont l'état a changé (via le champ `status`) qui permet de détecter les modifications dans les dépendances du symbole.

2.7 Vérification et résolution des symboles du module

Dans le noyau Linux, un symbole désigne une fonction ou une variable globale rendue visible afin qu'elle puisse être utilisée par d'autres parties du noyau ou par des modules chargeables. Les symboles constituent ainsi le mécanisme principal permettant la communication entre différents modules du noyau.

La table des symboles du noyau agit comme un répertoire central de toutes les fonctions, variables et constantes disponibles dans l'espace du noyau. Chaque symbole est associé à son adresse mémoire, ce qui permet au noyau et aux modules chargés de les localiser et d'y accéder efficacement.

Lorsqu'un module est compilé, il peut :

- définir ses propres symboles ;

- utiliser des symboles déjà fournis par le noyau ou par d'autres modules.

L'utilisation de `EXPORT_SYMBOL()` ou `EXPORT_SYMBOL_GPL()` dans un module demande au noyau d'ajouter le symbole à la table des symboles du noyau, le rendant accessible aux autres modules à l'exécution. Sans cette exportation, les autres modules ne pourront pas accéder au symbole [16].

La table des symboles du noyau peut être consultée via le fichier `/boot/System.map`, qui montre un instantané de tous les symboles connus du noyau à un moment donné. Elle est également accessible à travers `/proc/kallsyms` :

```
...
ffffffff8104c700 T pfn_range_is_mapped
ffffffff8104c750 T __pfx_devmem_is_allowed
ffffffff8104c760 T devmem_is_allowed
ffffffff8104c7d0 T __pfx_free_init_pages
...
```

Ici, la première colonne indique l'adresse du symbole, la deuxième colonne le type, et la troisième le nom du symbole. Pour plus de détails sur les différents types de symboles, on peut consulter la page Wikipédia [17].

Lorsque `insmod` charge un module, le noyau effectue une étape de résolution des symboles. Il vérifie que tous les symboles utilisés par le module existent et sont correctement adressés. Cette vérification est effectuée par la fonction `simplify_symbols()` :

```
/* Change all symbols so that st_value encodes the pointer directly. */
static int simplify_symbols(struct module *mod, const struct load_info *info)
{
    Elf_Shdr *symsec = &info->sechdrs[info->index.sym];
    Elf_Sym *sym = (void *)symsec->sh_addr;
    unsigned long secbase;
    unsigned int i;
    int ret = 0;
    const struct kernel_symbol *ksym;

    for (i = 1; i < symsec->sh_size / sizeof(Elf_Sym); i++) {
        const char *name = info->strtab + sym[i].st_name;

        switch (sym[i].st_shndx) {
        case SHN_COMMON:
            /* Ignore common symbols */
            if (!strcmp(name, "__gnu_lto", 9))
                break;

            /*
             * We compiled with -fno-common. These are not
             * supposed to happen.
             */
            pr_debug("Common symbol: %s\n", name);
            pr_warn("%s: please compile with -fno-common\n",
                    mod->name);
            ret = -ENOEXEC;
            break;

        case SHN_ABS:
            /* Don't need to do anything */
            pr_debug("Absolute symbol: 0x%08lx %s\n",
                    (long)sym[i].st_value, name);
            break;

        case SHN_LIVEPATCH:
            /* Livepatch symbols are resolved by livepatch */
            break;

        case SHN_UNDEF:
```

```

ksym = resolve_symbol_wait(mod, info, name);
/* Ok if resolved. */
if (ksym && !IS_ERR(ksym)) {
    sym[i].st_value = kernel_symbol_value(ksym);
    break;
}

/* Ok if weak or ignored. */
if (!ksym &&
    (ELF_ST_BIND(sym[i].st_info) == STB_WEAK ||
     ignore_undef_symbol(info->hdr->e_machine, name)))
    break;

ret = PTR_ERR(ksym) ?: -ENOENT;
pr_warn("%s: Unknown symbol %s (err %d)\n",
        mod->name, name, ret);
break;

default:
...
        secbase = info->sechdrs[sym[i].st_shndx].sh_addr;
        sym[i].st_value += secbase;
        break;
    }
}

return ret;
}

```

Ici, les symboles de livepatch sont marqués avec SHN_LIVEPATCH afin que le chargeur de modules puisse les identifier et les ignorer. Ils sont traités séparément. Pour plus d'informations, on peut consulter la section 2.8.

Avant cet appel, les symboles définis dans le module ont un champ `st_value` contenant uniquement un offset relatif à leur section. Les symboles importés du noyau contiennent `st_value = 0` et `st_shndx = SHN_UNDEF`.

La fonction `simplify_symbols()` met à jour les adresses dans la table des symboles du module. Ainsi, après son exécution, tous les symboles définis dans le module pointent vers leur adresse réelle en mémoire, tandis que les symboles importés sont résolus via `resolve_symbol_wait()` dans la table des symboles exportés du noyau.

Ainsi, si un symbole utilisé par le module n'existe pas ou n'est pas exporté, le chargement échoue :

```

root@vm:~$ insmod module_6_10.ko
module_6_10: Unknown symbol __x86_return_thunk (err -2)
insmod: can't insert 'module_6_10.ko': unknown symbol in module or invalid parameter

```

L'outil `readelf -s` permet d'afficher directement la table des symboles ELF du module.

```

root@LAPTOP-T02FKT7J:/mnt# readelf -s module_6_10.ko

Symbol table '.symtab' contains 56 entries:

```

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
...							
47:	0000000000000000	1024	OBJECT	GLOBAL	DEFAULT	23	__this_module
48:	0000000000000000	0	NOTYPE	GLOBAL	DEFAULT	18	__stop_alloc_tags
49:	0000000000000010	26	FUNC	GLOBAL	DEFAULT	4	cleanup_module
50:	0000000000000010	28	FUNC	GLOBAL	DEFAULT	2	init_module
51:	0000000000000000	0	NOTYPE	GLOBAL	DEFAULT	18	__start_alloc_tags
52:	0000000000000000	0	NOTYPE	GLOBAL	DEFAULT	UND	_prntk
53:	0000000000000000	16	FUNC	GLOBAL	DEFAULT	2	__pfx_init_module
54:	0000000000000000	16	FUNC	GLOBAL	DEFAULT	4	__pfx_cleanup_module
55:	0000000000000000	0	NOTYPE	GLOBAL	DEFAULT	UND	__x86_return_thunk

Chaque entrée de tableau de symboles est de type tel que défini par `struct ElfN_Sym` :

```
typedef struct
{
    Elf64_Word      st_name;           /* Symbol name (string tbl index) */
    unsigned char   st_info;          /* Symbol type and binding */
    unsigned char   st_other;         /* Symbol visibility */
    Elf64_Word      st_shndx;         /* Section index */
    Elf64_Addr      st_value;         /* Symbol value */
    Elf64_Xword     st_size;          /* Symbol size */
} Elf64_Sym;
```

Les champs dans la sortie `readelf` sont corrélés aux champs de la structure `struct ElfN_Sym`, comme décrit ci-dessous [18] :

- **Num** : index dans la table des symboles.
- **Ndx (st_shndx)** : indique la section dans laquelle le symbole est défini. Certaines valeurs spéciales existent :
SHN_ABS : le symbole possède une valeur absolue, inchangée après relocalisation.

SHN_COMMON : représente un bloc commun non encore alloué en mémoire. Dans ce cas-là, le champ `st_value` contient une contrainte d'alignement. L'éditeur de liens alloue l'espace mémoire à une adresse multiple de `st_value`. Le champ `st_size` indique la taille nécessaire pour ce symbole.

SHN_UNDEF : indique que le symbole est indéfini dans le fichier ELF courant. Il s'agit d'un symbole externe, affiché comme UND, qui sera résolu lors du chargement du module.

SHN_XINDEX : indique que l'index de section est trop grand pour être stocké dans `st_shndx`. L'index réel est alors stocké dans la section SHT_SYMTAB_SHNDX.

- **Value (st_value)** : valeur du symbole. Dans un module noyau, elle correspond généralement à un offset dans une section, ou vaut 0 pour un symbole externe. Pour SHN_COMMON, elle représente une contrainte d'alignement. Dans les exécutables et bibliothèques partagées, elle contient directement une adresse virtuelle.
- **Size (st_size)** : taille du symbole en octets.
- **Type (st_info)** : nature du symbole. Par exemple : STT_NOTYPE (non défini), STT_SECTION (section), STT_FILE (fichier source), STT_FUNC (fonction), STT_OBJECT (donnée).
- **Bind (st_info)** : portée du symbole. LOCAL indique un symbole interne au module, GLOBAL un symbole visible par le noyau ou d'autres modules, et WEAK un symbole global de priorité plus faible.
- **Vis (st_other)** : visibilité du symbole. DEFAULT signifie visibilité normale (dépend du binding), tandis que HIDDEN indique que le symbole n'est pas visible en dehors du binaire.
- **Name (st_name)** : index dans la table des chaînes (.strtab) permettant d'obtenir le nom du symbole.

Pour plus de détails, on peut consulter la documentation correspondante [19].

Le symbole `__this_module` mérite une attention particulière :

- il est de type OBJECT ;
- sa taille vaut 1024 octets ;
- il correspond à l'instance de `struct module` décrivant le module lui-même.

Dans le fichier `.ko`, `__this_module` apparaît avec `st_value = 0`. Cela signifie que la structure se trouve au début de sa section ELF (section 23 dans cet exemple). Son offset est donc nul à l'intérieur de cette section.

2.8 Vérification et application des relocalisations ELF

Le processus de relocalisation intervient lorsqu'un module noyau (.ko) est chargé dans le noyau Linux. Son rôle est de corriger les adresses présentes dans le code et les données du module afin qu'elles correspondent à leur emplacement réel en mémoire.

Le noyau Linux étant dynamique, un module peut être chargé ou déchargé à tout moment, sans redémarrage. Lors de la compilation, il est donc impossible de savoir à quelle adresse mémoire exacte le module sera placé. Si le module contenait des adresses fixes, il ne pourrait fonctionner que s'il était chargé exactement à cette adresse. Comme cela n'est jamais garanti, le compilateur produit à la place un code indépendant de la position (*Position Independent Code*, PIC).

Le principe est le suivant : dans le fichier .ko, les instructions ne contiennent pas encore les adresses définitives. À la place, le compilateur et l'éditeur de liens ajoutent des sections spéciales de relocalisation, par exemple :

- .rel.text ou .rela.text pour le code ;
- .rel.data ou .rela.data pour les données.

Ces sections indiquent au noyau où une adresse doit être corrigée et quel symbole est concerné.

Dans les programmes utilisateur compilés en PIC, cette correction passe souvent par une *Global Offset Table* (GOT). Le code ne référence alors pas directement la fonction ou la variable, mais une entrée dans cette table :

code → GOT → adresse réelle

Le chargeur remplit ensuite la GOT avec les bonnes adresses.

Les modules noyau utilisent rarement cette technique. Pour des raisons de performance et de simplicité, le noyau préfère généralement modifier directement le code du module lors du chargement :

instruction → adresse réelle

Autrement dit, la relocalisation écrit directement l'adresse finale dans l'instruction ou dans la donnée concernée. [20]

Après la résolution des symboles par `simplify_symbols()`, le noyau connaît l'adresse réelle de chaque symbole utilisé par le module. Les champs `st_value` de la table `.symtab` ne contiennent alors plus des offsets ou des zéros, mais les véritables adresses des fonctions et variables.

Le noyau applique ensuite les opérations de relocalisation lors du chargement du module via la fonction `apply_relocations()` :

```
static int apply_relocations(struct module *mod, const struct load_info *info)
{
    unsigned int i;
    int err = 0;

    /* Now do relocations. */
    for (i = 1; i < info->hdr->e_shnum; i++) {
        unsigned int infosec = info->sechdrs[i].sh_info;

        /* Not a valid relocation section? */
        if (infosec >= info->hdr->e_shnum)
            continue;

        if (!(info->sechdrs[infosec].sh_flags & SHF_ALLOC) &&
            (!infosec || infosec != info->index.pcpu))
            continue;

        if (info->sechdrs[i].sh_flags & SHF_RELA_LIVEPATCH)
            err = klp_apply_section_relocs(mod, info->sechdrs,
                                           info->secstrings,
                                           info->strtab,
```

```

                                info->index.sym, i,
                                NULL);

    else if (info->sechdrs[i].sh_type == SHT_REL)
        err = apply_relocate(info->sechdrs, info->strtab,
                                info->index.sym, i, mod);
    else if (info->sechdrs[i].sh_type == SHT_RELA)
        err = apply_relocate_add(info->sechdrs, info->strtab,
                                info->index.sym, i, mod);

    if (err < 0)
        break;
}
return err;
}

```

Le noyau parcourt l'ensemble des sections ELF du module et les traite si elles sont du types SHF_RELA-LIVEPATCH, SHT_REL et SHT_RELA.

Le flag SHF_RELA_LIVEPATCH est associé au mécanisme de correction dynamique du noyau (live patching). Le live patching du noyau Linux permet d'appliquer des correctifs de sécurité critiques ou importants à un noyau en cours d'exécution, sans nécessiter de redémarrage ni d'interruption de service [21].

Dans ce contexte, les relocations sont appliquées par la fonction `klp_apply_section_relocs()`, qui se situe dans `kernel/livepatch/core.c`. Voyons le code de cette fonction :

```

static int klp_write_section_relocs(struct module *pmod, Elf_Shdr *sechdrs,
                                    const char *shstrtab, const char *strtab,
                                    unsigned int symndx, unsigned int secndx,
                                    const char *objname, bool apply)
{
    int cnt, ret;
    char sec_objname[MODULE_NAME_LEN];
    Elf_Shdr *sec = sechdrs + secndx;
    cnt = sscanf(shstrtab + sec->sh_name, ".klp.rela.%55[^\n]",
                sec_objname);
    if (cnt != 1) {
        pr_err("section %s has an incorrectly formatted name\n",
                shstrtab + sec->sh_name);
        return -EINVAL;
    }

    if (strcmp(objname ? objname : "vmlinux", sec_objname))
        return 0;

    if (apply) {
        ret = klp_resolve_symbols(sechdrs, strtab, symndx,
                                sec, sec_objname);

        if (ret)
            return ret;

        return apply_relocate_add(sechdrs, strtab, symndx, secndx, pmod);
    }

    clear_relocate_add(sechdrs, strtab, symndx, secndx, pmod);
    return 0;
}

```

Il existe deux types de sections de relocations utilisées par le live patching du noyau. Les premières concernent les symboles du noyau (`vmlinux`) : `.klp.rela.vmlinux.*`. Ces relocations sont stockées dans le module de patch afin de permettre au code patché d'accéder à des symboles du noyau, même non exportés. Elles doivent être traitées très tôt pour garantir que les autres étapes d'initialisation puissent utiliser ces symboles.

Les secondes concernent les symboles des modules du noyau : `.klp.rela.{module}.*`. Cela permet au patch d'accéder aux symboles du module cible et de ses dépendances, qu'ils soient exportés ou non.

Les symboles de live patching sont des symboles référencés par les sections de relocation utilisées par le livepatch. Ils correspondent aux symboles utilisés par les nouvelles versions des fonctions appliquées aux objets patchés, dont les adresses ne peuvent pas être résolues par le chargeur de modules, car ils sont locaux ou non exportés.

En effet, le chargeur de modules ne résout que les symboles exportés, alors que certaines fonctions modifiées par le livepatch peuvent dépendre de symboles non exportés. Ces symboles sont alors résolus à l'aide de la fonction `klp_resolve_symbols()`.

Dans tous les cas, l'ensemble des symboles associés à une section de relocation du livepatch doit être résolu avant l'appel de la fonction `apply_relocate_add()`.

Les symboles de livepatch doivent être marqués avec `SHN_LIVEPATCH` afin que le chargeur de modules puisse les identifier et les ignorer. Les modules de livepatch conservent ces symboles dans leur table des symboles, laquelle est accessible via `module->symbtab` [22].

Les deux flags `SHT_REL` et `SHT_RELA` correspondent aux deux types d'entrées de relocalisation `REL` et `RELA`. Chaque entrée dans la table de relocalisation est essentiellement une structure en C de la forme suivante pour les relocalisations de type `REL` [23] :

```
typedef struct {
    Elf64_Addr    r_offset;    // Address
    Elf64_Xword   r_info;     // 32-bit relocation type; 32-bit symbol index
} Elf64_Rel;
```

et pour `RELA` :

```
typedef struct {
    Elf64_Addr    r_offset;    // Address
    Elf64_Xword   r_info;     // 32-bit relocation type; 32-bit symbol index
    Elf64_Sxword  r_addend;    // Addend
} Elf64_Rela;
```

Voyons les différents éléments de ces structures :

- **r_offset** : Ce champ indique l'emplacement où l'action de relocalisation doit être appliquée. Pour un fichier ELF, cette valeur correspond à un décalage en octets depuis le début de la section jusqu'à l'unité mémoire à modifier par la relocation. Pour un fichier exécutable ou une bibliothèque partagée, cette valeur correspond directement à l'adresse virtuelle de l'unité mémoire à corriger.
- **r_info** : Ce champ contient à la fois l'index dans la table des symboles utilisé pour effectuer la relocation, ainsi que le type de relocation à appliquer. Par exemple, une entrée de relocation associée à un appel de fonction contient l'index du symbole correspondant à la fonction appelée.

Si cet index vaut `STN_UNDEF`, cela signifie que le symbole est indéfini et que la valeur 0 est utilisée comme valeur de symbole lors du calcul de la relocation [24]. Ce champ est en réalité une valeur compacte qui encode ces deux informations dans un seul entier. Les macros ELF permettent d'extraire ou de construire ces champs :

```
#define ELF32_R_SYM(i)      ((i)>>8)
#define ELF32_R_TYPE(i)    ((unsigned char)(i))
#define ELF32_R_INFO(s,t)  (((s)<<8)+(unsigned char)(t))

#define ELF64_R_SYM(i)     ((i)>>32)
#define ELF64_R_TYPE(i)    ((i)&0xffffffffL)
#define ELF64_R_INFO(s,t)  (((s)<<32)+((t)&0xffffffffL))
```

Dans le cas des fichiers ELF 32 bits, les 8 bits de poids faible représentent le type de relocation, tandis que les 24 bits restants correspondent à l'index du symbole. Pour les fichiers ELF 64 bits les 32 bits de poids faible correspondent au type de relocalisation, tandis que les 32 bits de poids fort contiennent l'index du symbole.

Les types de relocalisation dépendent de l'architecture du processeur et indiquent comment calculer la valeur finale à écrire. On distingue notamment les types suivants :

- **R_X86_64_PC32 (ou similaire)** : correspond à un adressage *relatif au compteur de programme (PC)*. Ce type de relocalisation est souvent utilisé pour les sauts ou appels internes au module. Il indique que l'instruction doit sauter d'une certaine distance par rapport à sa position actuelle, et cette distance doit être recalculée lors du chargement du module.

- **R_X86_64_PLT32 (ou similaire)** : est généralement utilisé pour les appels de fonctions externes au module, par exemple des fonctions du noyau.
- **R_X86_64_64 (ou similaire)** : correspond à une adresse absolue. Cela signifie qu'une adresse mémoire complète doit être écrite directement à cet emplacement, souvent pour accéder à des données ou des fonctions situées dans le noyau ou dans d'autres modules.

Pour plus de détails sur les types de relocalisation, on peut consulter la source suivante [25].

- **r_addend** : C'est le décalage par rapport à l'adresse du symbole. Il est présent uniquement dans les relocalisations de type RELA et correspond à l'élément **r_addend** dans la structure et au champ **addend** dans la sortie readelf.

Comme on le voit, la seule différence entre les deux structures est que **Elfxx_Rela** contient un champ supplémentaire dédié en tant qu'addend de relocalisation. La raison d'utiliser un type d'entrée plutôt qu'un autre dépend généralement de l'architecture. Par exemple, dans **x86**, seule **Elf32_Rel** est utilisée, tandis que sur **x86_64** seule **Elf64_Rela** est utilisée.

Dans la suite, nous analysons le comportement des fonctions **apply_relocate()** et **apply_relocate_add()**, situées dans le fichier **linux/arch/x86/kernel/module.c**.

La fonction **apply_relocate()** traite les entrées de type **SHT_REL** :

```
int apply_relocate(Elf32_Shdr *sechdrs,
                  const char *strtab,
                  unsigned int symindex,
                  unsigned int relsec,
                  struct module *me)
{
    ..
    for (i = 0; i < sechdrs[relsec].sh_size / sizeof(*rel); i++) {
        /* This is where to make the change */
        location = (void *)sechdrs[sechdrs[relsec].sh_info].sh_addr + rel[i].r_offset;
        /* This is the symbol it is referring to. Note that all
           undefined symbols have been resolved. */
        sym = (Elf32_Sym *)sechdrs[symindex].sh_addr
            + ELF32_R_SYM(rel[i].r_info);

        switch (ELF32_R_TYPE(rel[i].r_info)) {
            case R_386_32:
                /* We add the value into the location given */
                *location += sym->st_value;
                break;
            case R_386_PC32:
            case R_386_PLT32:
                /* Add the value, subtract its position */
                *location += sym->st_value - (uint32_t)location;
                break;
            default:
                pr_err("%s: Unknown relocation: %u\n",
                       me->name, ELF32_R_TYPE(rel[i].r_info));
                return -ENOEXEC;
        }
    }
    return 0;
}
```

Lorsqu'une entrée de relocalisation est de type REL, la structure **ElfN_Rel** ne contient pas de champ **r_addend**. Dans ce cas, l'addend est déjà présent dans la zone mémoire ciblée par la relocation. Il s'agit donc d'un addend implicite [26].

La formule de base de relocalisation est alors :

$$\text{valeur finale} = A + S$$

où **A** correspond à l'addend et **S** à la valeur du symbole (**st_value**). Cela se traduit directement dans le code du noyau par :

```
*location += sym->st_value;
```

Dans le cas des relocalisations relatives (par exemple **R_386_PC32**), la position de l'instruction est également prise en compte :

$$\text{valeur finale} = S + A - P$$

où P représente la position de la zone mémoire à relocaliser, c'est-à-dire le décalage dans la section ou à l'adresse de l'unité mémoire à relocaliser.

On observe ainsi une correspondance directe avec l'implémentation du noyau :

```
*location += sym - > st_value - (uint32_t)location;
```

Pour les sections SHT_RELA, le noyau utilise la fonction `apply_relocate_add()`, qui délègue ensuite le traitement à `__write_relocate_add()` :

```
static int __write_relocate_add(Elf64_Shdr *sechdrs,
    const char *strtab,
    unsigned int symindex,
    unsigned int relsec,
    struct module *me,
    void *(*write)(void *dest, const void *src, size_t len),
    bool apply)
{
    ...
    for (i = 0; i < sechdrs[relsec].sh_size / sizeof(*rel); i++) {
        size_t size;

        /* This is where to make the change */
        loc = (void *)sechdrs[sechdrs[relsec].sh_info].sh_addr
            + rel[i].r_offset;

        /* This is the symbol it is referring to. Note that all
           undefined symbols have been resolved. */
        sym = (Elf64_Sym *)sechdrs[symindex].sh_addr
            + ELF64_R_SYM(rel[i].r_info);

        val = sym->st_value + rel[i].r_addend;

        switch (ELF64_R_TYPE(rel[i].r_info)) {
            case R_X86_64_NONE:
                continue; /* nothing to write */
            case R_X86_64_64:
                size = 8;
                break;
            ...
            case R_X86_64_PLT32:
                val -= (u64)loc;
                size = 4;
                break;
            case R_X86_64_PC64:
                val -= (u64)loc;
                size = 8;
                break;

            default:
                pr_err("%s: Unknown rela relocation: %llu\n",
                    me->name, ELF64_R_TYPE(rel[i].r_info));
                return -ENOEXEC;
        }

        if (apply) {
            if (memcmp(loc, &zero, size)) {
                pr_err("x86/modules: Invalid relocation target, existing value
                    ↪ is nonzero for type %d, loc %p, val %Lx\n",
                    (int)ELF64_R_TYPE(rel[i].r_info), loc, val);
                return -ENOEXEC;
            }
            write(loc, &val, size);
        }
    }
}
```

```

    } else {
        if (memcmp(loc, &val, size)) {
            pr_warn("x86/modules: Invalid relocation target, existing
↪ value does not match expected value for type %d, loc %p,
↪ val %Lx\n",
                    (int)ELF64_R_TYPE(rel[i].r_info), loc, val);
            return -ENOEXEC;
        }
        write(loc, &zero, size);
    }
}
return 0;
...
}

```

Lorsqu’une entrée de relocation est de type `RELA`, la structure `ElfN_Rela` contient un champ `r_addend`, appelé addend explicite.

La formule de base d’une relocation est la suivante :

$$\text{valeur finale} = S + A$$

ce qui correspond dans le code à :

```
val = sym->st_value + rel[i].r_addend;
```

Cette formule correspond au type `R_X86_64_64`. Toutefois, pour d’autres types de relocation, la formule diffère. Par exemple, pour les types `R_X86_64_PC32` et `R_X86_64_PLT32`, la valeur finale est calculée comme suit :

$$\text{valeur finale} = S + A - P$$

où `P` représente l’adresse de l’emplacement à modifier, ce qui correspond dans le code à l’instruction :

```
val -= (u64)loc;
```

Pour plus de détails sur les différentes formules de relocation, on peut consulter la source suivante [27]. Sur l’architecture `x86_64`, les modules utilisent principalement le format `RELA`.

Ainsi, le processus global de relocalisation repose sur deux étapes complémentaires :

- la résolution des symboles externes, qui associe chaque nom de symbole à une adresse réelle dans le noyau ;
- l’application des relocalisations, qui ajuste le code et les données du module en mémoire.

Testons un cas concret :

```

root@vm:~$ insmod module_6_10.ko
module_6_10: loading out-of-tree module taints kernel.
module: x86/modules: Invalid relocation target, existing value is nonzero for sec 24, idx 1,
↪ type 1, loc ffffffff0201420, val fffff0
insmod: can't insert 'module_6_10.ko': invalid module format

```

Avant d’écrire une relocalisation, le noyau calcule la valeur attendue (`val`) et vérifie le contenu actuel de la zone mémoire cible (`loc`).

Dans ce cas-là, l’erreur indique que la relocalisation concerne la section **24** du fichier ELF. L’adresse cible est `loc = ffffffff0201420`. À cette adresse, la valeur attendue correspondait à une zone initialement nulle, mais la mémoire contient déjà une valeur non nulle. Le noyau refuse alors l’écriture afin d’éviter de corrompre une zone déjà initialisée, ce qui entraîne l’échec du chargement du module.

On identifie la section concernée dans le fichier ELF :

```

root@LAPTOP-T02FKT7J:/mnt# readelf -S module_6_10.ko
Section Headers:
 [Nr] Name                Type              Address            Offset
      Size              EntSize          Flags  Link  Info  Align
...
 [24] .rela.gnu.linkonc     RELA              0000000000000000  0001b8f8
      0000000000000030  0000000000000018  I      40   23   8
...

```

Cette section est de type RELA et correspond à `.rela.gnu.linkonce.this_module`. Elle contient les relocalisations appliquées à la structure `struct module`, c'est-à-dire les corrections d'adresses nécessaires pour initialiser les champs internes du module lors de son chargement. Analysons des champs principaux :

- **Size = 0x30 (48 octets)** : taille totale de la section de relocalisation ;
- **EntSize = 0x18 (24 octets)** : taille d'une entrée `Elf64_Rela` ;
- **Nombre d'entrées** : $48/24 = 2$ relocalisations ;
- **Align = 8** : alignement mémoire requis pour les accès 64 bits.

Ainsi, cette section contient exactement deux opérations de correction mémoire. Vérifions le contenu des relocalisations :

```
root@LAPTOP-T02FKT7J:/mnt# readelf -r module_6_10.ko
...
Relocation section '.rela.gnu.linkonce.this_module' at offset 0x1b8f8 contains 2 entries:
  Offset          Info           Type           Sym. Value      Sym. Name + Addend
000000000130      003200000001 R_X86_64_64      000000000000010 init_module + 0
0000000003e0      003100000001 R_X86_64_64      000000000000010 cleanup_module + 0
...
```

Chaque entrée décrit une correction à appliquer dans la structure `struct module`. Le champ **Offset** indique la position (en octets) dans la structure `struct module` où la valeur calculée doit être écrite.

Le champ `st_value = 0x10` correspond à un offset dans la section `.text` du module. Lors du chargement, le noyau convertit cet offset en une adresse réelle dans l'espace mémoire alloué au module.

La section `.rela.gnu.linkonce.this_module` initialise deux pointeurs essentiels de la structure `struct module` : le pointeur d'initialisation (`init_module`) et le pointeur de sortie (`cleanup_module`). On observe dans le fichier ELF que :

- `init_module` est écrit à l'offset `0x130` (soit 304 en décimal) ;
- `cleanup_module` est écrit à l'offset `0x3e0` (soit 992 en décimal).

Ces valeurs correspondent à des positions dans l'image binaire de la structure `__this_module` générée lors de la compilation du module.

```
root@LAPTOP-T02FKT7J:/mnt# pahole -C module module_6_10.ko
struct module {
    enum module_state      state;          /*    0    4 */
    ...
    int                    (*init)(void);    /*  304    8 */
    ...
    void                    (*exit)(void);    /*  992    8 */
    ...
    /* size: 1024, cachelines: 16, members: 44 */
    /* sum members: 982, holes: 5, sum holes: 22 */
    /* padding: 20 */
    /* forced alignments: 1, forced holes: 1, sum forced holes: 8 */
} __attribute__((__aligned__(64)));
```

On constate donc que les offsets utilisés dans les relocalisations correspondent exactement aux champs `init` et `exit` dans la structure `struct module`.

3 Schémas du chargement et de la validation des modules noyau

3.1 Chaîne de chargement d'un module noyau après insmod

La commande `insmod` déclenche l'appel système `init_module` (ou `finit_module` dans le cas d'un chargement depuis un descripteur de fichier). La chaîne d'exécution côté noyau peut être résumée comme suit :

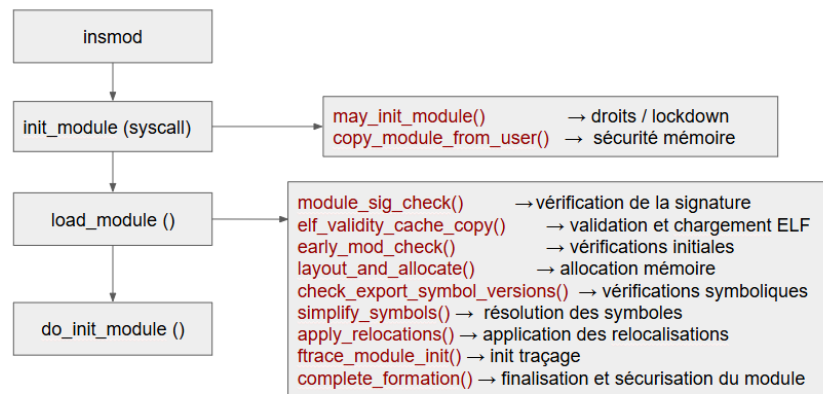


FIGURE 3 – Chaîne de chargement d'un module noyau après `insmod`.

Après l'exécution de la commande `insmod`, le chargement du module s'effectue via l'appel système `init_module()`. Cette fonction constitue le point d'entrée du noyau pour l'insertion d'un module et réalise uniquement deux vérifications préliminaires :

- **`may_init_module()`** : vérifie que le processus possède la capacité `CAP_SYS_MODULE` et que le chargement des modules n'est pas globalement désactivé ;
- **`copy_module_from_user()`** : copie l'image du module depuis l'espace utilisateur vers l'espace noyau et effectue des contrôles élémentaires sur les paramètres fournis.

Une fois ces vérifications initiales validées, la fonction `load_module()` est appelée. C'est dans cette fonction que se concentrent l'ensemble des vérifications de sécurité, de cohérence et de compatibilité du module.

Parmi les principales étapes de validation effectuées dans `load_module()`, on trouve :

- **`module_sig_check()`** : vérification de la signature cryptographique du module ;
- **`elf_validity_cache_copy()`** : validation de la structure ELF et mise en cache des informations nécessaires ;
- **`early_mod_check()`** : vérifications précoces, incluant la comparaison du `vermagic`, la cohérence de la structure `struct module`, le contrôle de la présence du module dans une éventuelle liste noire, ainsi que la détection d'un module déjà chargé portant le même nom ;
- **`layout_and_allocate()`** : calcul du layout mémoire et allocation des zones nécessaires au module ;
- **`check_export_symbol_versions()`** : parcourt la section `__versions` du module et compare le CRC de chaque symbole importé (`_printf`, `module_layout`, etc.) avec le CRC du symbole exporté par le noyau. Si un seul CRC diffère, le chargement est interrompu avec un message du type « `disagrees about version of symbol` » ;
- **`simplify_symbols()`** : résout les symboles du module en remplaçant les références symboliques par leurs adresses réelles dans le noyau. Cette étape parcourt la table des symboles du module et associe chaque symbole externe (fonctions ou variables du noyau) à son adresse via la table des symboles exportés. Le champ `st_value` de chaque symbole est alors mis à jour avec l'adresse résolue, permettant aux étapes suivantes (notamment les relocalisations) d'utiliser directement des adresses valides.

- **apply_relocations()** : applique les relocalisations ELF. Les références vers les fonctions et données du noyau sont remplacées par leurs adresses réelles en mémoire. Par exemple, un appel à `_printk` dans le code du module est transformé en appel vers l'adresse effective de `_printk` dans le noyau ;
- **ftrace_module_init()** : initialise les structures nécessaires au traçage dynamique du module lorsque `CONFIG_FTRACE` est activé. Cela permet ensuite à des outils comme `ftrace` ou à des hooks internes du noyau de tracer les fonctions du module ;
- **complete_formation()** : vérifie l'absence de conflits de symboles exportés via `verify_exported_symbols()`, applique les protections mémoire (RO/NX) et prépare le module avant son activation.

Ainsi, `init_module()` ne réalise que des contrôles d'accès et de transfert mémoire, tandis que la majorité des vérifications critiques sont centralisées dans `load_module()`, qui constitue le cœur du mécanisme de validation des modules noyau.

3.2 Structure interne d'un module compilé avec `CONFIG_MODVERSIONS`

La figure suivante présente l'organisation simplifiée des principales sections d'un module noyau Linux compilé avec l'option `CONFIG_MODVERSIONS`. Elle met en évidence les emplacements des informations utilisées lors de la vérification de compatibilité du module par le noyau.

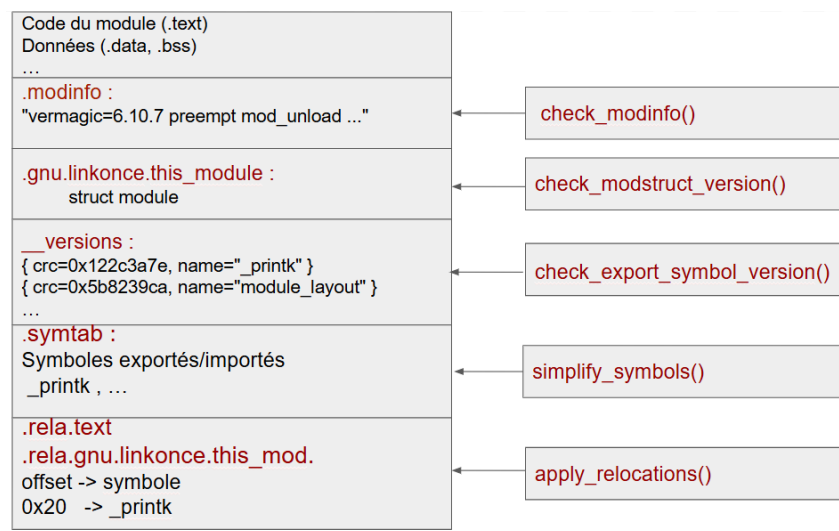


FIGURE 4 – Structure interne simplifiée d'un module noyau Linux compilé

- **__versions** : contient un tableau associant chaque symbole importé par le module à son CRC. Cette section est utilisée lorsque `CONFIG_MODVERSIONS` est activé afin de vérifier la compatibilité binaire entre le module et le noyau courant ;
- **.modinfo** : contient les informations textuelles du module, notamment la chaîne `vermagic`, mais aussi le nom de l'auteur, la licence, la description et les dépendances éventuelles ;
- **.gnu.linkonce.this_module** : contient l'instance de la structure `struct module` générée pour le module. Cette structure stocke notamment le nom du module, son état, les pointeurs vers les fonctions d'initialisation et de sortie ;
- **.symtab** : table des symboles définis dans le module (fonctions, variables globales). Elle est utilisée pour la résolution des symboles lors de l'application des relocalisations par `apply_relocations()` ;
- **.rela.text** : table de relocalisation pour la section `.text` (code du module). Chaque entrée indique comment remplacer les symboles externes par leurs adresses réelles en mémoire. Cette section est essentielle pour que les appels à des fonctions du noyau soient correctement résolus ;
- **.rela.gnu.linkonce.this_module** : table de relocalisation pour la section `.gnu.linkonce.this_module`. Elle ajuste les adresses internes de la structure `struct module`.

Les sections présentes dans le module peuvent être affichées avec les commandes `readelf -S` ou `objdump -h`. L'outil `pahole` permet d'afficher le contenu détaillé de la structure `struct module`, si celui-ci a été compilé avec les informations de débogage.

3.3 Arbre des appels des fonctions des vérifications du noyau pour le chargement d'un module

L'arbre présenté ci-dessous illustre les fonctions de vérification du noyau impliquées dans le processus de chargement d'un module. Seules les fonctions participant directement à la sécurité, la validation et la cohérence des données sont affichées. Les autres opérations internes (allocations mémoire, gestion de symboles, initialisation des modules, etc.) ont été volontairement ignorées afin de ne conserver que l'enchaînement des vérifications critiques.

```
+-- init_module(umod, len, uargs)
|   +-- may_init_module()
|   |   -> vérifie CAP_SYS_MODULE et modules_disabled
|   |
|   +-- copy_module_from_user()
|   |   -> vérifie sécurité avant et après chargement
|   |
|   +-- load_module()
|   |   +-- module_sig_check()
|   |   |   -> vérifie signature du module
|   |   |
|   |   +-- elf_validity_cache_copy()
|   |   |   +-- elf_validity_cache_sechdrs()
|   |   |   |   -> vérifie header ELF
|   |   |   +-- elf_validity_cache_secstrings()
|   |   |   |   -> vérifie table de noms des sections
|   |   |   +-- elf_validity_cache_index_info()
|   |   |   |   -> cache index .modinfo et nom module
|   |   |   +-- elf_validity_cache_strtab()
|   |   |   |   -> cache table de noms des symboles et vérifie offsets
|   |   |
|   |   +-- early_mod_check()
|   |   |   +-- blacklisted()
|   |   |   |   -> vérifie si le module est dans une liste noire
|   |   |   +-- rewrite_section_headers()
|   |   |   |   -> réécrit headers de sections si nécessaire
|   |   |   +-- check_modstruct_version()
|   |   |   |   -> vérifie la compatibilité de la structure struct module
|   |   |   +-- check_modinfo()
|   |   |   |   -> vérifie le champs .modinfo du module
|   |   |   +-- module_patient_check_exists()
|   |   |   |   -> vérifie doublons dans liste des modules (mutex)
|   |   |
|   |   +-- check_export_symbol_versions()
|   |   |   -> vérifie versions des symboles exportés
|   |   |
|   |   +-- simplify_symbols()
|   |   |   -> résout les symboles de .symtab
|   |   |
|   |   +-- apply_relocations()
|   |   |   -> applique les relocalisations ELF
|   |
|   +-- do_init_module()
|       -> exécute le module chargé
```

4 Analyse d'un outil de modification de `vermagic` et CRC de module

L'outil disponible sur GitHub [28] permet de modifier le champ `vermagic` d'un module `.ko`. Son fonctionnement repose sur une manipulation directe du fichier ELF du module, sans nécessiter la recompilation du noyau.

4.1 Fonctionnement général

Le programme charge le module en mémoire à l'aide de `mmap` avec des droits de lecture et d'écriture. Il détecte automatiquement le format ELF (32 ou 64 bits), puis initialise les structures correspondantes pour accéder directement aux sections internes du module.

L'outil analyse principalement deux sections : `.modinfo` et `__versions`. La section `.modinfo` contient les informations textuelles du module, dont la chaîne `vermagic`. Pour récupérer la valeur `vermagic` du noyau courant, le programme lit la première ligne du fichier `modules.dep` correspondant à la version du noyau, extrait le chemin d'un module présent sur le système, puis lit sa section `.modinfo`. Il compare ensuite cette valeur avec celle du module cible et peut la remplacer par une nouvelle chaîne afin de rendre le module compatible avec un noyau différent.

La section `__versions` contient les CRC des symboles du noyau utilisés par le module. L'outil permet de modifier ces CRC, voire de les recalculer automatiquement à partir de fichiers de référence tels que `Module.symvers` ou `/boot/sysver-<version>.gz`.

Enfin, les modifications effectuées en mémoire sont écrites dans le fichier via `msync`, puis la mémoire allouée est libérée avec `munmap`.

Lors de l'exécution de `vermagic -x`, plusieurs fonctions sont appelées pour récupérer la version du noyau, lire la chaîne `vermagic` du module, la mettre à jour et ajuster les CRC correspondants. Le schéma ci-dessous résume ces appels et leurs actions principales.

```
+++ set_vermagic_and_crc("module.ko")
|
+++ __get_kernel_version(kernel_key_path, &kernel_version_num)
|   -> lit la version du noyau courant depuis "/proc/version"
|
+++ get_os_vermagic(kernel_key_path, &__os_vermagic)
|   -> obtient le vermagic du noyau
|   |
|   +++ __get_one_ko_path_from_dep()
|   |   -> ouvre "/lib/modules/<version>/modules.dep",
|   |       récupère le chemin d'un module .ko du noyau courant
|   |
|   +++ read_vermagic_str_from_ko()
|   |   -> extrait la chaîne située après "vermagic="
|   |
+++ __set_vermagic("module.ko", __os_vermagic)
|   -> remplace la valeur vermagic du module cible
|
+++ __update_crc_to_ko_file(kernel_key_path, "module.ko")
|   -> remplace CRC du module par le CRC du noyau
|
|   +++ set_check_version_key_crc()
|   |
|   |   +++ find_section("__versions")
|   |   |   -> localise la table des CRC des symboles pour le module cible
|   |   |
|   |   +++ get_os_sym_key_crc(kernel_key_path, symbol)
|   |   |   -> recherche le CRC attendu pour chaque symbole
|   |   |       dans "/lib/modules/<version>/build/Module.symvers" ou "/boot/sysver-<version>.gz"
|   |   |
|   |   +++ get_os_sym_key_crc_from_symvers()
|   |   |   -> lit le CRC du symbole puis remplace celui du module
```

4.2 Preuve de concept : cas sans l'option CONFIG_MODVERSIONS

On compile `vermagic.c` :

```
gcc -static -Wall -o vermagic vermagic.c -lelf -lz
```

Pour simplifier l'analyse, les premiers tests sont réalisés avec un noyau compilé sans l'option `CONFIG_MODVERSIONS`. Dans cette configuration, seules les vérifications liées à la chaîne `vermagic` sont effectuées, sans contrôle des CRC présents dans la section `__versions`.

Pour désactiver cette option, il suffit de modifier la configuration du noyau via `menuconfig` :

```
Enable loadable module support --->
[ ] Module versioning support
```

Cette option correspond au symbole de configuration `CONFIG_MODVERSIONS`.

Pour inspecter un module compilé, on peut utiliser la commande `readelf` afin de lister les sections ELF et filtrer celle de type `.gnu.linkonce` :

```
root@LAPTOP-T02FKT7J:/mnt# readelf -SW module_6_10.ko | grep .gnu.linkonce
[22] .gnu.linkonce.this_module PROGBITS 0000000000000000 000240 000400 00 WA 0 0 64
```

Chaque champ de cette ligne peut être interprété comme suit :

- **[22]** : numéro de la section dans la table des sections ELF.
- **.gnu.linkonce.this_module** : nom de la section, qui contient la structure `struct module` du module compilé.
- **PROGBITS** : type de section, contenant des données binaires “program bits”.
- **0000000000000000** : adresse virtuelle (VMA) prévue pour le chargement en mémoire ; 0 car le module n'est pas encore chargé.
- **000240** : offset de la section dans le fichier ELF, soit $0x240 = 576$ octets.
- **000400** : taille de la section, $0x400 = 1024$ octets. Cela correspond à l'espace réservé pour `struct module` dans cette compilation.
- **WA** : flags de section. W signifie que la section est modifiable par le noyau, A signifie que la mémoire doit être allouée pour cette section lors du chargement.
- **64** : alignement de la section en mémoire, ici 64 octets, ce qui correspond à l'alignement requis pour `struct module`.

Afin d'isoler uniquement le comportement lié à `vermagic`, les tests sont également réalisés avec des modules possédant une structure `struct module` de taille identique.

```
root@LAPTOP-T02FKT7J:/mnt/# readelf -SW module_6_19.ko | grep .gnu.linkonce
[14] .gnu.linkonce.this_module PROGBITS 0000000000000000 0000c0 000400 00 WA 0 0
↪ 64
root@LAPTOP-T02FKT7J:/mnt/# readelf -SW module_6_18.ko | grep .gnu.linkonce
[22] .gnu.linkonce.this_module PROGBITS 0000000000000000 000240 000400 00 WA 0 0
↪ 64
root@LAPTOP-T02FKT7J:/mnt# readelf -SW module_6_11.ko | grep .gnu.linkonce
[22] .gnu.linkonce.this_module PROGBITS 0000000000000000 000240 000400 00 WA 0 0
↪ 64
root@LAPTOP-T02FKT7J:/mnt# readelf -SW module_6_10.ko | grep .gnu.linkonce
[22] .gnu.linkonce.this_module PROGBITS 0000000000000000 000240 000400 00 WA 0 0
↪ 64
```

Dans les quatre cas, la taille de la section vaut `0x400`, soit 1024 octets. On peut donc considérer que la structure `struct module` a la même taille dans ces différentes versions du noyau. L'offset de la section dans le fichier ELF varie (`0x0000c0` ou `0x000240`), mais cela n'a pas d'impact : seule sa taille importe ici. Les différences observées par la suite proviendront donc uniquement de la chaîne `vermagic`.

4.2.1 Vermagic module = Vermagic noyau

On commence par vérifier la version du noyau courant ainsi que la valeur `vermagic` du module à charger :

```

root@vm:~$ uname -r
6.19.10
root@vm:~$ modinfo module_6_18.ko
filename:      module_6_18.ko
author:        Anonymous Developer
description:    Test module
license:        GPL
version:        1.0
srcversion:     749D2AC5E7268FCD5DF202C
depends:
vermagic:      6.18.20 preempt mod_unload

```

La commande `modinfo` montre que le module a été compilé pour le noyau 6.18.20, alors que le noyau en cours d'exécution est la version 6.19.10. Le chargement du module échoue donc immédiatement :

```

root@vm:~$ insmod module_6_18.ko
module_6_18: version magic '6.18.20 preempt mod_unload ' should be '6.19.10 preempt mod_unload
↪ '
insmod: can't insert 'module_6_18.ko': invalid module format

```

L'outil `vermagic` est ensuite exécuté afin de remplacer la chaîne `vermagic` du module par celle du noyau courant. Pour rendre les étapes plus explicites, quelques messages d'affichage ont été modifiés dans le programme :

```

root@vm:~$ ./vermagic -x module_6_18.ko
[+] os_version : <6.19.10>
[+] os_module_path :
↪ </lib/modules/6.19.10/kernel/drivers/thermal/intel/x86_pkg_temp_thermal.ko>

Section name:                .modinfo
[+] os_vermagic : <6.19.10 preempt mod_unload >

Section name:                .modinfo
[-] Old value => vermagic=6.18.20 preempt mod_unload
[+] New value => vermagic=6.19.10 preempt mod_unload
[!] Old len vermagic = 27
[!] New len vermagic = 27

Not any section named: __versions
Module compiled without CONFIG_MODVERSIONS, skipping CRC update

```

Le programme commence par récupérer la version du noyau courant (6.19.10), puis sélectionne un module déjà présent dans `/lib/modules/6.19.10/kernel/drivers/thermal/intel/` afin d'extraire sa valeur `vermagic`. Cette valeur est ensuite utilisée pour remplacer celle du module cible dans la section `.modinfo`.

L'absence de la section `__versions` indique que le module a été compilé avec `CONFIG_MODVERSIONS` désactivé. Aucune mise à jour des CRC n'est donc nécessaire. Enfin, le chargement du module est tenté à nouveau :

```

root@vm:~$ insmod module_6_18.ko
module_6_18: loading out-of-tree module taints kernel.
module: Module inserted!
root@vm:~$ lsmod module_6_18.ko
module_6_18 12288 0 - Live 0xffffffffa0000000 (0)

```

La présence du message `Module inserted!` confirme que le module a été correctement chargé après la modification de la chaîne `vermagic`.

4.2.2 Vermagic module > Vermagic noyau

Essayons maintenant avec un autre module compilé pour une version plus ancienne du noyau : `small`

```

root@vm:~$ uname -r
6.19.10
root@vm:~$ modinfo module_6_10.ko
filename:      module_6_10.ko
author:        Anonymous Developer

```

```

description:    Test module
license:       GPL
version:       1.0
srcversion:    749D2AC5E7268FCD5DF202C
depends:
vermagic:      6.10.7 preempt mod_unload
root@vm:~$ insmod module_6_10.ko
module_6_10: version magic '6.10.7 preempt mod_unload ' should be '6.19.10 preempt mod_unload
↪ '
insmod: can't insert 'module_6_10.ko': invalid module format

```

Dans ce cas, l'outil ne parvient pas à modifier la valeur `vermagic`. Pour comprendre cette limitation, l'affichage du programme a été enrichi afin de montrer la longueur de l'ancienne et de la nouvelle chaîne :

```

root@vm:~$ ./vermagic -x module_6_10.ko
[+] os_version : <6.19.10>
[+] os_module_path :
↪ </lib/modules/6.19.10/kernel/drivers/thermal/intel/x86_pkg_temp_thermal.ko>

Section name:                .modinfo
[+] os_vermagic : <6.19.10 preempt mod_unload >

Section name:                .modinfo
[-] Old value => vermagic=6.10.7 preempt mod_unload
[!] Old len vermagic = 26
[!] New len vermagic = 27
[!] Length of the new specified vermagic overflow

Not any section named: __versions
Module compiled without CONFIG_MODVERSIONS, skipping CRC update

```

Le programme refuse de modifier la chaîne lorsque la nouvelle valeur est plus longue que l'ancienne. En effet, la fonction `set_vermagic()` remplace directement la chaîne dans la section `.modinfo` sans réallouer d'espace supplémentaire dans le fichier ELF. Afin d'éviter d'écraser les données suivantes dans la section, elle vérifie que la longueur de la nouvelle chaîne est inférieure ou égale à celle déjà présente.

Dans cet exemple, la chaîne `"6.10.7 preempt mod_unload "` contient 26 caractères, alors que la chaîne `"6.19.10 preempt mod_unload "` en contient 27. Le remplacement est donc refusé et le module reste inchangé.

Une nouvelle tentative de chargement confirme que le module est toujours rejeté par le noyau :

```

root@vm:~$ insmod module_6_10.ko
module_6_10: version magic '6.10.7 preempt mod_unload ' should be '6.19.10 preempt mod_unload
↪ '
insmod: can't insert 'module_6_10.ko': invalid module format

```

Cette expérience met en évidence une limite importante de l'outil : il ne fonctionne que si la nouvelle chaîne `vermagic` possède une longueur inférieure ou égale à celle de la chaîne d'origine. Dans le cas contraire, une modification plus complexe du fichier ELF serait nécessaire, par exemple en agrandissant la section `.modinfo` et en mettant à jour les offsets associés.

4.2.3 Vermagic module < Vermagic noyau

Dans cet exemple, testons le cas inverse, c'est-à-dire lorsque la nouvelle chaîne `vermagic` est plus courte que celle présente dans le module. Le noyau courant est ici en version 6.10.7, alors que le module a été compilé pour le noyau 6.18.20 :

```

root@vm:~$ uname -r
6.10.7
root@vm:~$ modinfo module_6_18.ko
filename:      module_6_18.ko
author:        Anonymous Developer
description:    Test module
license:       GPL
version:       1.0
srcversion:    749D2AC5E7268FCD5DF202C

```

```
depends:
vermagic:      6.18.20 preempt mod_unload
```

Au premier essai, le noyau refuse de charger le module, indiquant une incompatibilité de version magic : la chaîne du module 6.18.20 preempt mod_unload ne correspond pas à celle du noyau 6.10.7 preempt mod_unload.

```
root@vm:~$ insmod module_6_18.ko
module_6_18: version magic '6.18.20 preempt mod_unload ' should be '6.10.7 preempt mod_unload
↪ '
insmod: can't insert 'module_6_18.ko': invalid module format
```

Ensuite, l'outil est utilisé pour remplacer la valeur de vermagic dans le module :

```
root@vm:~$ ./vermagic -x module_6_18.ko
[+] os_version : <6.10.7>
[+] os_module_path :
↪ </lib/modules/6.10.7/kernel/drivers/thermal/intel/x86_pkg_temp_thermal.ko>

Section name:                .modinfo
[+] os_vermagic : <6.10.7 preempt mod_unload >

Section name:                .modinfo
[-] Old value => vermagic=6.18.20 preempt mod_unload
[+] New value => vermagic=6.10.7 preempt mod_unload
[!] Old len vermagic = 27
[!] New len vermagic = 26

Not any section named: __versions
Module compiled without CONFIG_MODVERSIONS, skipping CRC update
```

Contrairement au cas précédent, la modification réussit car la nouvelle chaîne contient moins de caractères que l'ancienne. La fonction set_vermagic() remplace la chaîne puis complète l'espace restant avec des octets nuls afin de conserver la taille initiale de la section .modinfo. Le module peut alors être chargé avec succès :

```
root@vm:~$ insmod module_6_18.ko
module_6_18: loading out-of-tree module taints kernel.
module: Module inserted!
root@vm:~$ lsmod
module_6_18 12288 0 [permanent], Live 0xffffffffa0000000 (0)
```

Une tentative de déchargement échoue :

```
root@vm:~$ rmmod module_6_18.ko
rmmod: remove 'module_6_18': Device or resource busy
```

Cependant, une différence importante apparaît. Le module est marqué comme [permanent] dans la sortie de lsmod. Cela signifie que le noyau considère qu'il ne peut jamais être déchargé. En réalité, le module possède une fonction de sortie, mais après modification du module et du CRC, le noyau interprète la structure struct module différemment : le champ mod->exit est lu comme nul.

Cette logique est visible dans la fonction print_unload_info() du noyau Linux :

```
#ifdef CONFIG_MODULE_UNLOAD
static inline void print_unload_info(struct seq_file *m, struct module *mod)
{
    struct module_use *use;
    int printed_something = 0;

    seq_printf(m, " %i ", module_refcount(mod));

    /*
     * Always include a trailing , so userspace can differentiate
     * between this and the old multi-field proc format.
     */
    list_for_each_entry(use, &mod->source_list, source_list) {
```

```

        printed_something = 1;
        seq_printf(m, "%s", use->source->name);
    }

    if (mod->init && !mod->exit) {
        printed_something = 1;
        seq_puts(m, "[permanent]");
    }

    if (!printed_something)
        seq_puts(m, "-");
}
#else /* !CONFIG_MODULE_UNLOAD */
static inline void print_unload_info(struct seq_file *m, struct module *mod)
{
    /* We don't know the usage count, or what modules are using. */
    seq_puts(m, " - -");
}
#endif /* CONFIG_MODULE_UNLOAD */

```

Cette fonction affiche les informations liées au déchargement d'un module, notamment le compteur d'utilisation, la liste des modules dépendants et le statut `[permanent]` si le module n'a pas de fonction de sortie ou si cette fonction est interprétée comme absente. Dans ce cas, le module est considéré comme permanent et ne peut pas être déchargé via `rmmod`, même si sa fonction de sortie existe dans la version originale du module.

Pour mieux comprendre ce comportement, examinons en détail la structure `struct module` d'un module compilé pour le noyau courant :

```

root@LAPTOP-T02FKT7J:/mnt# pahole -C module module_6_10.ko
struct module {
    enum module_state      state;          /*    0    4 */
    ...
    int                    (*init)(void);  /*  304    8 */
    ...
    struct list_head       source_list;    /*  960   16 */
    struct list_head       target_list;    /*  976   16 */
    void                    (*exit)(void);  /*  992    8 */
    atomic_t               refcnt;         /* 1000    4 */

    /* size: 1024, cachelines: 16, members: 44 */
    /* sum members: 982, holes: 5, sum holes: 22 */
    /* padding: 20 */
    /* forced alignments: 1, forced holes: 1, sum forced holes: 8 */
} __attribute__((__aligned__(64)));

```

Comparons maintenant avec la structure `struct module` du module compilé pour une autre version du noyau, que nous avons utilisé précédemment :

```

root@LAPTOP-T02FKT7J:/mnt# pahole -C module module_6_18.ko
struct module {
    enum module_state      state;          /*    0    4 */
    ...
    int                    (*init)(void);  /*  304    8 */
    ...
    struct list_head       source_list;    /*  976   16 */
    struct list_head       target_list;    /*  992   16 */
    void                    (*exit)(void);  /* 1008    8 */
    atomic_t               refcnt;         /* 1016    4 */

    /* size: 1024, cachelines: 16, members: 44 */
    /* sum members: 998, holes: 5, sum holes: 22 */
    /* padding: 4 */
    /* forced alignments: 1, forced holes: 1, sum forced holes: 8 */
} __attribute__((__aligned__(64)));

```


On observe que, bien que la taille totale de `struct module` reste identique (1024 octets) pour les deux modules, certains champs sont décalés dans la version compilée pour le noyau 6.18.

Plus précisément, la fonction `init` pointe correctement vers `init_module` avec le même offset (304) dans les deux versions. Cependant, la fonction `exit`, qui devrait pointer vers `cleanup_module`, se trouve à l'offset 1008 dans le module cible alors que le noyau 6.10 s'attend à la lire à l'offset 992.

En conséquence, lorsque le noyau lit `mod->exit` à l'offset 992, il obtient une valeur incorrecte, interprétée comme `NULL`. Cela rend le test `(mod->init && !mod->exit)` vrai et le module est alors affiché comme `[permanent]`.

Le marquage `[permanent]` ne reflète pas l'absence réelle d'une fonction de sortie dans le module, mais plutôt une incompatibilité de structure entre la version du noyau et celle pour laquelle le module a été compilé. Même si la fonction `exit` existe, le noyau la croit absente à cause de ce décalage structurel.

4.3 Preuve de concept : cas avec l'option `CONFIG_MODVERSIONS`

Lorsque l'option `CONFIG_MODVERSIONS` est activée, chaque module compilé génère une section `__versions` contenant les symboles importés et leurs CRC associés. Cette section permet au noyau de vérifier la compatibilité des symboles utilisés par le module avec ceux exportés par le noyau.

Nous pouvons examiner la section `__versions` du module pour voir les symboles importés et leurs CRC :

```
root@LAPTOP-T02FKT7J:mnt# readelf -x __versions module_6.18.ko
Hex dump of section '__versions':
0x00000000 7e3a2c12 00000000 5f707269 6e746b00 ~:,..._printk.
0x00000010 00000000 00000000 00000000 00000000 .....
0x00000020 00000000 00000000 00000000 00000000 .....
0x00000030 00000000 00000000 00000000 00000000 .....
0x00000040 ca39825b 00000000 5f5f7838 365f7265 .9.[...__x86_re
0x00000050 7475726e 5f746875 6e6b0000 00000000 turn_thunk.....
0x00000060 00000000 00000000 00000000 00000000 .....
0x00000070 00000000 00000000 00000000 00000000 .....
0x00000080 d2cad11c 00000000 6d6f6475 6c655f6c .....module_l
0x00000090 61796f75 74000000 00000000 00000000 aayout.....
0x000000a0 00000000 00000000 00000000 00000000 .....
0x000000b0 00000000 00000000 00000000 00000000 .....
```

Chaque entrée de cette section correspond à une structure `modversion_info`. Cette structure a une taille fixe de 64 octets. Les 8 premiers octets contiennent le CRC associé au symbole et les 56 octets restants contiennent le nom du symbole. Les noms de symboles dépassant cette taille ne sont pas pris en charge dans ce format. Ils nécessitent l'utilisation du mécanisme de *modversions étendus* (*extended modversions*). [11]

Par exemple, la première ligne peut être décomposée ainsi :

- `0x00000000` : offset de l'entrée dans la section `__versions` ;
- `7e3a2c12 00000000` : CRC du symbole sur 8 octets ;
- `5f707269 6e746b00` : nom du symbole encodé en ASCII hexadécimal.

Comme l'architecture `x86_64` est little-endian, la valeur `7e3a2c12 00000000` correspond au CRC `0x122c3a7e`. De même, la suite hexadécimale `5f707269 6e746b00` correspond à la chaîne ASCII `_printk`.

Les trois entrées présentes dans ce module sont donc :

- `__printk` avec le CRC `0x122c3a7e` ;
- `__x86_return_thunk` avec le CRC `0x5b8239ca` ;
- `module_layout` avec le CRC `0x1cd1cad2`.

Le symbole `module_layout` est particulièrement important. son CRC dépend notamment de la disposition interne de `struct module`. Si la définition de cette structure change entre deux versions du noyau, le CRC de `module_layout` change également, même si la taille totale de la structure reste identique.

Les CRC des symboles exportés par le noyau sont stockés dans le fichier `Module.symvers`. Pour `_printk` et `__x86_return_thunk`, on retrouve bien les mêmes CRC que ceux présents dans la section `__versions` :

```
root@LAPTOP-T02FKT7J:/mnt# cat Module.symvers | grep printk
...
0x122c3a7e      _printk vmlinux EXPORT_SYMBOL
...

root@LAPTOP-T02FKT7J:/mnt# cat Module.symvers | grep __x86_return_thunk
0x5b8239ca      __x86_return_thunk      vmlinux EXPORT_SYMBOL
```

```
root@LAPTOP-T02FKT7J:/mnt# cat Module.symvers | grep module_layout
0x1cd1cad2      module_layout      vmlinux EXPORT_SYMBOL
```

On constate que les CRC présents dans `__versions` correspondent exactement à ceux listés dans `Module.symvers`, ce qui permet au noyau de vérifier que le module utilise bien les symboles corrects.

4.3.1 Vermagic module = Vermagic noyau

Vérifions la version du noyau et du module :

```
oot@vm:~$ uname -r
6.19.10
root@vm:~$ modinfo module_6_18.ko
filename:      module_6_18.ko
author:        Anonymous Developer
description:    Test module
license:        GPL
version:        1.0
srcversion:     749D2AC5E7268FCD5DF202C
depends:
vermagic:       6.18.20 preempt mod_unload modversions
```

En exécutant notre outil `vermagic`, nous obtenons :

```
root@vm:~$ ./vermagic -x module_6_18.ko
[+] os_version : <6.19.10>
[+] os_module_path :
↳ </lib/modules/6.19.10/kernel/drivers/thermal/intel/x86_pkg_temp_thermal.ko>

Section name:                .modinfo
[+] os_vermagic : <6.19.10 preempt mod_unload modversions >

Section name:                .modinfo
[-] Old value => vermagic=6.18.20 preempt mod_unload modversions
[+] New value => vermagic=6.19.10 preempt mod_unload modversions
[!] Old len vermagic = 39
[!] New len vermagic = 39

Section name:                __versions
[-]Old value => _printk:      0x122C3A7E
[+]New value => _printk:      <0x122c3a7e>
[-]Old value => __x86_return_thunk: 0x5B8239CA
[+]New value => __x86_return_thunk: <0x5b8239ca>
[-]Old value => module_layout: 0x1CD1CAD2
[+]New value => module_layout: <0x007d6e31>
```

Enfin, après modification, le module peut être chargé dans le noyau :

```
root@vm:~$ ./vermagic -x module_6_18.ko
root@vm:~$ insmod module_6_18.ko
module: Module inserted!
root@vm:~$ lsmod
module_6_18 12288 0 - Live 0xffffffffa0000000 (0)
```

4.3.2 Vermagic module != Vermagic noyau

On commence par mettre en commentaire la fonction `__set_vermagic` dans `set_vermagic_and_crc` utilisée par l'option `-x` de `vermagic.c`. Ainsi, lors de l'exécution du programme `vermagic`, seule la modification des CRC est effectuée, et la chaîne `vermagic` reste inchangée.

On peut vérifier la version du noyau courant :

```
root@vm:~$ uname -r
6.10.7
root@vm:~$ insmod module_6_11.ko
```

```
module_6_11: disagrees about version of symbol module_layout
insmod: can't insert 'module_6_11.ko': invalid module format
```

En exécutant `vermagic` avec l'option `-x`, on constate que seuls les CRC changent :

```
root@vm:~$ ./vermagic -x module_6_11.ko
[+] os_version : <6.10.7>
[+] os_module_path :
↳ </lib/modules/6.10.7/kernel/drivers/thermal/intel/x86_pkg_temp_thermal.ko>

Section name:                                .modinfo
[+] os_vermagic : <6.10.7 preempt mod_unload modversions >

Section name:                                __versions
[-] Old value => _printk:                    0x122C3A7E
[+] New value => _printk:                    <0x122c3a7e>
[-] Old value => __x86_return_thunk:         0x5B8239CA
[+] New value => __x86_return_thunk:         <0x5b8239ca>
[-] Old value => module_layout:              0x2909D5FB
[+] New value => module_layout:              <0x1bba6ad5>
root@vm:~$ modinfo module_6_11.ko
filename:      module_6_11.ko
author:        Anonymous Developer
description:    Test module
license:       GPL
version:       1.0
srcversion:    749D2AC5E7268FCD5DF202C
depends:
vermagic:      6.11.5 preempt mod_unload modversions
```

Comme l'option `CONFIG_MODVERSIONS` est activée, le noyau ignore la vérification de la chaîne `vermagic` et se base uniquement sur les CRC des symboles importés. Ainsi, le module est correctement chargé même si la chaîne `vermagic` ne correspond pas exactement à celle du noyau.

```
root@vm:~$ insmod module_6_11.ko
module: Module inserted!
root@vm:~$ lsmod
module_6_11 12288 0 - Live 0xffffffffa0000000 (0)
```

Références

- [1] Joseph Simon. Exploiting linux capabilities : Cap_sys_module. <https://redfoxsec.com/blog/exploiting-linux-capabilities-capsysmodule-exploits/>, September 2023. Redfox Security – Pen Testing Services.
- [2] Config_module_sig : Module signature verification. https://cateee.net/lkddb/web-lkddb/MODULE_SIG.html#google_vignette.
- [3] Kernel module signing facility. <https://www.kernel.org/doc/html/v4.17/admin-guide/module-signing.html>.
- [4] Signed kernel module support. https://wiki.gentoo.org/wiki/Signed_kernel_module_support.
- [5] Pkcs 11. https://en.wikipedia.org/wiki/PKCS_11.
- [6] Pem, der, crt, and cer : X.509 encodings and conversions. <https://www.ssl.com/guide/pem-der-crt-and-cer-x-509-encodings-and-conversions/>.
- [7] pkcs7_verify.c. https://github.com/hizilla/linux-stable/blob/master/crypto/asymmetric_keys/pkcs7_verify.c.
- [8] Signing a kernel and modules for secure boot. https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/managing_monitoring_and_updating_the_kernel/signing-a-kernel-and-modules-for-secure-boot.
- [9] https://www.kernelconfig.io/config_secondary_trusted_keyring.
- [10] Module versioning. <https://docs.kernel.org/kbuild/modules.html#module-versioning>.
- [11] <https://blog.pmhahn.de/linux-kernel-symbol-versioning/>.

- [12] genksyms. <https://ccrma.stanford.edu/planetccrma/man/man8/genksyms.8.html>.
- [13] genksyms.c. <https://github.com/torvalds/linux/blob/master/scripts/genksyms/genksyms.c>.
- [14] Understanding the crc32 header : A deep dive into cyclic redundancy check. <https://medium.com/@mark.benly/understanding-the-crc32-header-a-deep-dive-into-cyclic-redundancy-check-3b34d31585c7>.
- [15] What is crc32? a guide to quick checksums for data integrity. <https://tooladex.com/blog/crc32-calculator>.
- [16] Kernel symbol table in linux. <https://embeddedprep.com/understanding-kernel-symbol-table-in-linux/>.
- [17] System.map. <https://en.wikipedia.org/wiki/System.map>.
- [18] https://compilepeace.github.io/BINARY_DISSECTION_COURSE/ELF/SYMBOLS/SYMBOLS.html.
- [19] <https://refspecs.linuxbase.org/elf/gabi4+/ch4.symtab.html>.
- [20] Linux kernel module relocation process. <https://www.linkedin.com/pulse/linux-kernel-module-relocation-process-david-zhu-tdb3c/>.
- [21] <https://www.redhat.com/en/topics/linux/what-is-linux-kernel-live-patching>.
- [22] <https://www.kernel.org/doc/html/v6.19/livepatch/module-elf-format.html>.
- [23] <https://gotplt.org/posts/relocations-fantastic-symbols-but-where-to-find-them.html>.
- [24] <https://refspecs.linuxbase.org/elf/gabi4+/ch4.reloc.html>.
- [25] <https://stackoverflow.com/questions/6093547/what-do-r-x86-64-32s-and-r-x86-64-64-relocation-mean>.
- [26] <https://reverseengineering.stackexchange.com/questions/17346/what-is-an-addend>.
- [27] <https://intezer.com/blog/executable-and-linkable-format-101-part-3-relocations/>.
- [28] Linux-kernel-module-vermagic. <https://github.com/YioHoohoo/Linux-Kernel-Module-vermagic>.